LECTURE NOTES · MODELS, 2026

Decision Models

Companion notes to the slides

Models that answer typed questions with calibrated probabilities instead of generating text: the primitives (choice, score, noul, box), what calibration means and how it is trained for, selective prediction, thresholds, conformal sets and cascades, the October 2026 landscape around TypeSafe's Jev and its open-weight relatives, the failure modes, the independent evidence, and how to put one inside an agent. One section per slide, every term defined, and a reading list for each part.

For advanced undergraduates and beginning graduate students

October 2026

These notes follow the deck *Decision Models* (32 slides) slide by slide. Numbers quoted from a paper were checked against that paper; numbers from vendor documentation, model cards and blog posts are named as such and marked †; numbers marked "derived" were computed from quoted ones and the formulas are given; curves the slides label "illustrative" are sketches of a mechanism, not data. The subject is five weeks old at the time of writing (Jev was released on September 15, 2026); prices, limits, model lists and benchmark numbers will have changed by the time you read this, and the reading list's primary papers will not.

Contents

How to use these notes **3**

1 Overview (slides 1–4) **3**
1.1 Slide 1: Title: Decision Models . . . 3
1.2 Slide 2: A decision model answers typed questions with probabilities . . . 4
1.3 Slide 3: Why now: an agent loop is full of small judgments . . . 6
1.4 Slide 4: A short history: the parts are old, the packaging is new . . . 7

2 Primitives (slides 5–9) **8**
2.1 Slide 5: Four primitives: choice, score, noul, box . . . 8
2.2 Slide 6: Choice: a distribution over your options, plus one number . . . 9
2.3 Slide 7: Score: ordered levels, an expected level, and a spread . . . 11
2.4 Slide 8: Noul and Box: a single probability, and four edges . . . 12
2.5 Slide 9: State and questions: facts go in, judgments are asked . . . 13

3 Calibration (slides 10–13) **14**
3.1 Slide 10: Calibration: the probability has to mean something . . . 14
3.2 Slide 11: Proper scoring rules make honesty the best policy . . . 16
3.3 Slide 12: Do language models know what they know? Until post-training . . . 17
3.4 Slide 13: Four ways to get a probability out of a language model . . . 18

4 Training (slides 14–16) **19**
4.1 Slide 14: What is known about how a decision model is trained . . . 19
4.2 Slide 15: RLHF, RLVR, RLCD: three objectives, three kinds of model . . . 21
4.3 Slide 16: Calibration after the fact: temperature, Platt, isotonic . . . 22

5 Selection (slides 17–20) **23**
5.1 Slide 17: Selective prediction: answer when sure, hand off the rest . . . 23
5.2 Slide 18: Thresholds are a cost decision, not a model setting . . . 24
5.3 Slide 19: Conformal prediction: sets with a coverage guarantee . . . 26
5.4 Slide 20: Cascades and fan-out: the cheap model decides who pays . . . 27

6 Landscape (slides 21–23) **28**
6.1 Slide 21: Jev: the product that named the category . . . 28
6.2 Slide 22: The landscape, October 2026: one category, many insides . . . 29
6.3 Slide 23: Open weights: four ways to build one, all on Hugging Face . . . 31

7 Limits (slides 24–25) **33**
7.1 Slide 24: Jaggedness: where a decision model is rough . . . 33
7.2 Slide 25: What a calibrated probability cannot tell you . . . 34

8 Evaluation (slides 26–27) **35**
8.1 Slide 26: Evaluating a decision model: six questions, six numbers . . . 35
8.2 Slide 27: What the independent evidence says, five weeks in . . . 37

9 Practice (slides 28–30) 38
9.1 Slide 28: One decision, as it goes over the wire 38
9.2 Slide 29: Writing questions that work 39
9.3 Slide 30: Build your own: three recipes, in increasing effort 40

10 Wrap-up (slides 31–32) 42
10.1 Slide 31: Summary: what to take from this lecture 42
10.2 Slide 32: Sources 42

A Notation 44

B Formulas at a glance 45

C Exercises 46

D Reading list 48

Index of terms 58

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the arithmetic written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Conventions. A *decision model* is a model that, given a state and a typed question, returns a probability distribution over a fixed set of answers rather than text; the deck's running example is TypeSafe's Jev, and the vendor's own vocabulary (System One, Choice, Score, Noul, Box, confidence, RLCD) is used where it is the only vocabulary there is, with a dagger when a claim rests on vendor material. p is a probability returned by the model, $p_{\max}$ the largest of a Choice distribution, n the number of options or levels, y a label, τ a threshold, α a conformal miscoverage level. "Calibrated" always means the population property of Section 3, never a property of one answer.

A reading plan for a single lecture: Sections 1–2 (what a decision model is and the four primitives) take about a quarter of the session; Sections 3–4 (calibration, proper scoring rules, training) a third; Section 5 (selective prediction, thresholds, conformal sets, cascades) a quarter; Sections 6–9 (the landscape, the limits, the evidence, and practice) the rest, and the last two can be read as a case study. The exercises in Appendix C are mostly arithmetic on the deck's own formulas and can be set as homework. Appendix D is the reading list, section by section, and matches the paper archive that accompanies these notes.

1 Overview (slides 1–4)

1.1 Slide 1: Title: Decision Models

Models that answer typed questions with calibrated probabilities instead of generating text.

What the slide shows A small request on the right of the title: a one-line state ("My card was charged twice."), a question (which team owns this?) and four options (billing, bug, account, other); an arrow; and four bars, 0.62, 0.21, 0.12 and 0.05, with a confidence of 0.49 beneath them. That picture is the whole subject. Nothing is generated: the model is handed a finite set of answers and returns how much probability it puts on each. Everything else in the lecture is about what that distribution means, how a model can be trained to make it mean something, what you can build with it, and where it breaks.

Details The earlier decks in this series were about generative models: the transformer, pretraining, scaling, post-training, sparsity and precision, and the runtime that serves them. All of those models produce text one token at a time, and when an application needs a *decision* (which queue, is this harmful, does the citation support the claim, how urgent) it has had to prompt a generator, parse the text that comes back, and hope. A decision model is the alternative that became a product in September 2026: a model trained to take a *state* (the content to judge) and one or more *typed questions* (a choice among options, a yes/no, a rating on an ordered scale, a box in an image) and to return, for each question, a probability distribution over its answers and nothing else.

The ten footer stages of the deck are Overview, Primitives, Calibration, Training, Selection, Landscape, Limits, Evaluation, Practice and Wrap-up. The first two say what the thing is. Calibration and Training are the science: what it means for the returned probabilities to be honest, why that is hard for a generative model, and what training objective produces it. Selection is what you do with an honest probability: answer when sure, defer when not, set thresholds from costs, wrap the answers in sets with guarantees, and put a cheap decision in front of an expensive generation. Landscape is the October 2026 market: the first product, Jev, from TypeSafe AI; the twenty-odd competitors that appeared within five weeks; and the open-weight reimplementations on Hugging Face that show how such a model can be built. Limits and Evaluation are the honest part: what the vendor admits, what independent testers found, and how to measure a model whose product is a probability. Practice is a worked request, a style guide for questions, and three recipes for building your own.

Two conventions run through the deck and these notes. First, a dagger (†) marks any number or claim whose source is vendor documentation, a model card, a blog post, a community benchmark or press coverage rather than a paper; the subject is young and most of what is known about the products is of this kind. Second, "derived" marks numbers computed from quoted ones, and the formula is always given. The confidence of 0.49 on the title slide is derived: $(0.62 - 0.25)/0.75$, the Choice confidence formula of slide 6.

Terms

decision model A model that takes a state and a typed question and returns a probability distribution over a fixed set of answers, with no generated text. The subject of the lecture.

System One model TypeSafe's name for its decision models, after Kahneman's fast, automatic System 1; Jev is "the first System One model" in the vendor's words.

Jev TypeSafe AI's decision model, released September 15, 2026 (version 1.13) and updated October 14 (1.14); the running example of the deck.

state The content a question is asked about: a string, a JSON object or an array. Slide 9.

typed question A question whose answer shape is fixed in advance: Choice, Score, Noul or Box. Slides 5–8.

confidence A single number from 0 to 1 that summarizes how concentrated the returned distribution is; derived from the probabilities, not a separate judgment. Slides 6–7.

calibrated Of a set of probabilities: among the answers given probability p, about a fraction p are correct. A population property. Section 3.

Intuition

A generative model is a writer you ask for an opinion and then have to read. A decision model is a panel of judges who hold up numbered cards. The cards are the product; the lecture is about when to believe the numbers.

1.2 Slide 2: A decision model answers typed questions with probabilities

Same transformer lineage, different contract: a distribution over options you supplied, no text to parse.

What the slide shows Two API calls side by side. On the left, a generative call: a chat-completions request asking a frontier model to classify a support ticket in one word, and the reply "Billing.": text, to be parsed, with no probability attached. On the right, a decision call to the System One endpoint: a state object holding the ticket, a question of type `choice` with the instruction "Which team owns `ticket`?" and

four criteria, and the reply: the chosen option, a probability for each option, and a confidence. Two panels below: what is different (the output is a distribution over a set you supplied; the probabilities are the contract; many questions per call) and what it is not (it does not write, explain, plan, call tools or reason step by step).

Details The request shape is worth reading closely because it is the whole interface. The JSON body of a decision request has three parts†: a `model` (an alias such as `jev-latest` or a pinned version such as `jev-1.13.0`); a `state`; and a map of `questions`, each with a `type`, free-text `instructions`, and `criteria` whose form depends on the type (a list of options for Choice, a list of ordered levels for Score, a description of the yes and no cases for Noul). The response carries, for each question id, an `answers` entry with the typed result: for a Choice, the `choice` (the argmax), the `probabilities` over every option, and the `confidence`. The example's confidence, 0.49, is derived from the probabilities by the formula of slide 6; it is not an independent assessment.

Three things change relative to the generative call. The first is mechanical: because the answer is one of the options you listed, there is nothing to parse and no format to drift; the model cannot refuse, add a caveat, or answer a different question. The second is semantic: the probabilities are what the model is trained to get right (Sections 3–4), so "0.62 billing" is meant to be read as "about 62% of tickets that look like this to the model are billing tickets", a claim the generative reply cannot make. The third is economic: the vendor prices input at $0.042 per million tokens with output free, answers in about 100 ms, and evaluates every question in a request independently in the same pass†, so a batch of twenty questions about one document costs the document's tokens once.

What the model is not is as important. It does not generate, so it cannot explain its answer, cannot reason through a multi-step problem, cannot call a tool, and cannot propose an option you did not list. Kahneman's System 1 is the vendor's metaphor: fast, automatic, pattern-based judgment, with the slow, deliberate System 2 work left to reasoning models and, mostly, to the application's own code. The deck's position is that this is calibrated classification as a service: the pieces (classifiers trained with proper scoring rules, logprob readouts of generative models, reward and guard models) are old, and slide 4 says so; what is new is the generality of the question interface, the training objective aimed at calibration, and the price.

Terms

System One API TypeSafe's HTTP endpoint (`POST /v1/systemone`) that takes a model, a state and a map of questions and returns typed answers; OpenRouter exposes the same shape as its decisions API. Slide 9.

Choice A question that selects one option from a supplied set; the answer is the argmax, a probability per option, and a confidence. Slide 6.

criteria The answer set of a question: the options of a Choice, the levels of a Score, or the yes/no definitions of a Noul.

instructions The free-text question, which can refer to parts of the state by backticked path (`` `ticket` ``).

argmax The option with the highest probability; what the `choice` field reports.

generative model A model that produces text token by token; the chat and reasoning models of the earlier decks, used here as the contrast.

output tokens Tokens a model generates; a decision model generates none, which is why the vendor can price them at zero.

Intuition

The generative call asks "what do you think?" and gets an essay; the decision call asks "which of these, and how sure?" and gets a vector. Everything the application wanted was the vector.

1.3 Slide 3: Why now: an agent loop is full of small judgments

Most of what an agent decides is a classification, a yes/no or a rating, and a generator is an expensive way to get one.

What the slide shows An agent loop drawn as six boxes: request, guard, route, retrieve, generate, verify, with a return arrow (act, or escalate when confidence is low). Under each decision box is the primitive it uses: a Noul for the guard (jailbreak? harmful?), a Choice for routing (intent, tool, model), a Score for passage relevance in retrieval, a Noul for verification (is the claim cited? does the reply follow policy?). Only the generate box is a reasoning model. Two panels: why a generator is the wrong tool for those judgments (cost and latency, no probability, fragile format) and where it is the right tool (hard cases, explanations, anything not on the list).

Details Count the decisions in a single turn of a production agent. Before the model sees the request: is it an injection attempt, is it harmful, which product area, which queue, which model tier. During retrieval: which of forty candidate passages are relevant, in what order. After generation: is every claim supported by a retrieved passage, does the reply violate a policy, did the tool call succeed, is the result worth remembering. Each of these is a classification, a yes/no, or a rating, and each has traditionally been done either by a purpose-built classifier (expensive to build, narrow) or by prompting a generative model (expensive to run, hard to trust).

The slide's cost arithmetic is derived from two list prices: a frontier model at $5 per million input tokens (the price the vendor's own cascade cookbook quotes for gpt-5.5[†]) against $0.042 for the decision model, a ratio of about 120 on input tokens alone; the generator also bills output tokens ($30 per million for the same model) and takes seconds where the decision model takes about 100 ms[†]. The "no probability" point is the scientific one and occupies Section 3: asking a chat model how sure it is produces numbers that are overconfident and clustered (Xiong et al. 2023), and its token log-probabilities, which were calibrated after pretraining, are not after preference tuning (the GPT-4 report's Figure 8). The "fragile format" point is the practical one: text must be parsed, and a judge prompted to compare two answers prefers whichever comes first and whichever is longer (Zheng et al. 2023), so even the label is not stable.

The right panel keeps the balance. A decision model cannot explain its verdict, cannot work through a case that needs several steps, and cannot choose anything outside the list. The design that follows is the one Section 5 develops: the cheap, calibrated model decides the easy majority and *says* which cases are hard, and those go to the expensive model or to a person. The probability is what makes that split possible; without it there is no principled place to draw the line.

Terms

agent loop The program that repeatedly calls a model, executes the tool calls it emits, appends the results, and stops at a final answer; the subject of the runtime deck's last section.

guard A check applied to inputs or outputs for safety or policy; traditionally a guard model such as Llama Guard, here a battery of Noul questions.

routing Choosing a code path, a queue, a specialist model or a tool for a request; a Choice question.

verification Checking a generated output for a specific failure mode: an unsupported claim, a policy violation, a malformed tool call; a Noul question per failure mode.

Noul TypeSafe's name for a yes/no question whose answer is the probability that the answer is yes. Slide 8.

Score A question answered on an ordered scale of 2–10 levels; the answer is the expected level, a probability per level, and a confidence. Slide 7.

escalation Handing a case to a person or a stronger model when the decision model's confidence is below a threshold. Section 5.

Intuition

An agent is a river of small yes/no and which-one questions with an occasional essay in the middle. Using the essay-writer for the yes/no questions is like hiring a novelist to sort mail.

1.4 Slide 4: A short history: the parts are old, the packaging is new

Calibrated classifiers, option logprobs, reward models, judges and guards all led here; the 2026 product is the category.

What the slide shows A timeline from 2017 to 2026 with eight events, alternating above and below the line: Guo et al. (2017) on the overconfidence of modern networks and temperature scaling; BERT-era classifiers and Desai and Durrett's measurement of their calibration (2019–20); MMLU's convention of scoring options by token log-probability (2020); InstructGPT's reward models and Kadavath et al.'s "language models (mostly) know what they know" (2022); LLM-as-a-judge, G-Eval, Llama Guard and FrugalGPT cascades (2023); RewardBench, Prometheus 2, RouteLLM and RL for verbalized confidence (2024); RLCR and "why language models hallucinate" (2025); and Jev on September 15, 2026, with OpenRouter's guide listing 22 decision models from 13 makers a month later. Two panels: what is old and what is new.

Details The point of the timeline is that no single idea in a decision model is new, and that the reader should recognize each one. *Calibrated classification* is the oldest: a softmax classifier trained by cross-entropy is trained on a proper scoring rule (slide 11), and Niculescu-Mizil and Caruana (2005) catalogued which classical learners come out calibrated (logistic regression, bagged trees, the neural networks of that era) and which need a post-hoc correction (boosted trees, SVMs). Guo et al. (2017) found that the deep networks of 2016 had lost that property and that one fitted temperature restores most of it. *Reading a decision out of a language model* began as an evaluation convenience: MMLU (Hendrycks et al. 2020) scores a model by the log-probability it assigns to the letter of the correct option, and Robinson et al. (2022) showed that presenting the options jointly and reading the symbol is what makes large models good at it. *Reward models* (InstructGPT, 2022) are decision models in all but name: one forward pass over a prompt and a response, a scalar out, trained on pairwise preferences. *Guard models* (Llama Guard, 2023) are classifiers wearing a generative costume: they emit "safe" or "unsafe" as a token. *LLM-as-a-judge* (Zheng et al. 2023) and G-Eval (Liu et al. 2023) made the generative route explicit, with G-Eval already using the probability-weighted score that the Score primitive formalizes. *Cascades and routers* (FrugalGPT 2023, RouteLLM 2024) are the economics: a cheap model in front, an expensive one behind, a confidence deciding who answers. And *training for calibration with RL* (Lin et al. 2022 by fine-tuning; SaySelf, LACIE and Leng et al. in 2024; Rewarding Doubt and RLCR in 2025) is the academic track that the vendor's RLCD most resembles (slide 14).

What is new is the packaging: one general model that accepts any question in a few fixed shapes, with the answer set supplied in the request rather than fixed at training time; calibration as the stated training objective rather than a post-hoc fix; a price and latency that let it be called a million times in the background; and a market category, with an endpoint name ("decisions") and a product name ("System One") that competitors

adopted within weeks. Whether the general model is as good as a classifier you train yourself on your own labels is the empirical question of slides 26–27, and the honest answer in October 2026 is: on narrow tasks, roughly a draw; on breadth and on questions you did not anticipate, the general model wins.

Terms

temperature scaling Dividing a model's logits by a fitted scalar T before the softmax; the simplest post-hoc calibration (Guo et al. 2017). Slide 16.

logprob readout Obtaining a probability for each option by reading the log-probability the generative model assigns to the option's token; the MMLU convention. Slide 13.

reward model A model trained on pairwise human preferences to score a response with a scalar; the RLHF component that is itself a decision model.

LLM-as-a-judge Using a generative model, by prompting, to grade or compare outputs; Zheng et al. (2023).

G-Eval Liu et al. (2023): an LLM evaluation method whose score is the probability-weighted sum over rating tokens rather than the argmax.

guard model A classifier for unsafe content, often implemented as a fine-tuned generative model that emits a verdict token (Llama Guard, ShieldGemma).

cascade A chain of models from cheap to expensive in which each case goes to the next model only if the current one is not confident enough. Slide 20.

router A model or rule that sends each request to one of several models based on predicted difficulty or preference. Slide 20.

RLCD "Reinforcement learning for calibrated decisions", TypeSafe's undisclosed training method; not the RLCD of Yang et al. (2023), which is contrastive distillation for alignment. Slide 14.

Intuition

If you have trained a classifier with cross-entropy, read an MMLU score, or used a reward model, you have already used a decision model. The 2026 product is those three things with a shared question interface and a calibration objective, sold by the token.

2 Primitives (slides 5–9)

2.1 Slide 5: Four primitives: choice, score, noul, box

Every question has a fixed answer shape; the shape decides what comes back and what confidence means.

What the slide shows Four cards: Choice (one option from a set of 1–255; returns the choice, a probability per option, a confidence), Score (a position on 2–10 ordered levels; returns the expected level, a probability per level, a confidence), Noul (a yes/no judgment; returns the probability of yes, with no separate confidence), and Box (where something is in an image; returns four edges as fractions of the image, and a confidence). Each card has an example question. Two panels: how to choose a shape, and the status of each primitive in October 2026 (Box and images are a preview behind an opt-in header).

Details The primitives are the vocabulary of the whole interface, and their names are the vendor's[†]; the ideas under them are standard. A *Choice* is a multiclass classification with the classes named in the request.

A *Score* is ordinal regression: the levels have an order and distances, and the model returns a distribution over them from which an expected level is computed. A *Noul* is binary classification with a single probability. A *Box* is localization: a bounding box expressed as fractions of the image's width and height, measured from the top-left corner. Everything the lecture says about calibration applies to all four, because each returns probabilities (Box returns a confidence rather than a distribution, and is the least mature).

Choosing a shape is choosing the question's logic†. If exactly one of several known categories should win, it is a Choice; the options must be exhaustive, because the model has to put its mass somewhere (slide 6). If the question is whether a property is present, with a strength, it is a Noul; if several properties might be present at once, it is several Nouls, not one Choice. If the answer is a position on a rubric (severity, relevance, quality), it is a Score, and the fact that the returned score can fall between levels is a feature. A broad judgment ("is this a good support reply?") is several of these (is it accurate, is it polite, does it answer the question, does it follow policy), asked in one call and combined in code (slide 9).

The status panel records what was true at the time of writing†: Choice, Score and Noul are production primitives on both Jev 1.13 and 1.14; images (PNG or JPEG, 280–1,568 pixels on each side, at most 16 MiB) may be placed in the state, in a question's instructions or in its criteria, but only behind the request header `TypeSafe-Danger-Zone-Images: true`; Box questions need Jev 1.14 and the same header; and the vendor's own words for images and Box are that they "aren't yet up to our standards to call ready for production", with a warning that the format, the answers and the limits may change between releases. The header name is the vendor's way of making the opt-in explicit.

Terms

Box A question that locates something in an image; the answer is `top`, `bottom`, `left`, `right` as fractions of the image's height and width from the top-left corner, plus a confidence. Jev 1.14 only, preview. Slide 8.

ordinal regression Predicting a position on an ordered scale whose levels have distances; the classical name for what a Score question does.

multiclass classification Assigning one of *n* named classes; a Choice question.

binary classification Assigning yes or no with a probability; a Noul question.

exhaustive options An option set that covers every case the state might present, so that the forced distribution has somewhere honest to put its mass; add "none of these" or "uncertain" when it does not.

opt-in header An HTTP header that must be sent to enable a preview feature (`TypeSafe-Danger-Zone-Images` for images and Box, `TypeSafe-Danger-Zone-Long-Context` for 262k contexts); the vendor's signal that the feature is not covered by its production claims.

Intuition

Four shapes of answer: pick one, rate it, yes or no, point at it. If the question you want to ask does not fit one of the four, it is probably two questions.

2.2 Slide 6: Choice: a distribution over your options, plus one number

The answer is the argmax; the probabilities are the real output; confidence is how far the top sits above an even split.

What the slide shows A bar chart of the title slide's example: billing 0.62, bug 0.21, account 0.12, other

0.05. Under it the confidence formula and its evaluation: $(0.62-0.25)/0.75 = 0.49$ for $n = 4$. A paragraph on reading the number and on its blind spot (only $p_{\max}$ counts), and a panel on designing the option set: make it exhaustive, describe each option, do not rely on long flat lists, do not trust the order, and note that a two-option Choice is a Noul.

Details The vendor's confidence for a Choice is[†]

$$\text{confidence} = \frac{p_{\max} - \frac{1}{n}}{1 - \frac{1}{n}}, \qquad n = \text{number of options, } p_{\max} = \text{probability of the chosen option.}$$

It is 0 when the distribution is uniform ($p_{\max} = 1/n$) and 1 when all mass sits on one option, and it is linear in between. For the example, $n = 4$ gives $(0.62-0.25)/(1-0.25) = 0.37/0.75 = 0.493$, reported as 0.49. The docs' own illustration of the blind spot is the pair $(0.6, 0.3, 0.1)$ and $(0.6, 0.2, 0.2)$: both have confidence $(0.6-1/3)/(2/3) = 0.4$, although the first has a strong runner-up and the second does not. The docs list the top probability itself and the ratio of the top to the second probability as alternatives (the entropy of the distribution is a third), and say their formula is "one reasonable way to summarize a distribution, not the only one"[†]. The lecture's advice is to pick one summary, calibrate your thresholds against it (slide 18), and never change it without re-tuning.

The option-set panel collects the design rules that matter most in practice. *Exhaustive*: the model must distribute probability over the options it is given, so if the true answer is not among them the mass lands on the nearest wrong one with whatever confidence the shape produces; the vendor's self-consistency recipe adds an explicit "uncertain" outcome for exactly this reason[†]. *Described*: the docs recommend giving each option a `what`, a `not_for` and `examples`; the errors live between confusable neighbours, and the descriptions are where the boundary is drawn. *Not too long*: the API accepts up to 255 options, but a long flat list is a weak question; the hierarchical-classification cookbook searches a taxonomy level by level with a beam of width 3 over the path probabilities, scored by the geometric mean of the edge probabilities so that shallow and deep leaves compare fairly[†]. *Order-sensitive*: first-option bias is on the vendor's own jaggedness list and in every independent test (slide 24); shuffle the options, or ask twice in two orders and average. *Two options*: a Choice with two options is a Noul with a confidence attached, and the formulas agree: for $n = 2$, $(p_{\max} - \frac{1}{2})/\frac{1}{2} = 2p_{\max} - 1 = |2p - 1|$.

Terms

uniform distribution Equal probability $1/n$ on each of n options; confidence 0 by the formula.

margin The difference between the top two probabilities; an alternative confidence that sees the runner-up.

entropy $-\sum_i p_i \log p_i$; an alternative confidence that sees the whole distribution, maximal at uniform.

what / not_for / examples The three fields the docs recommend for describing each option: what it covers, what belongs elsewhere, and instances.

beam search over Choice probabilities Classifying through a hierarchy by asking a Choice at each node and keeping the K best paths by the geometric mean of their edge probabilities; the vendor's hierarchical-classification cookbook ($K = 3$).

first-option bias A tendency to favour the option listed first, independent of content. Slide 24.

Intuition

Confidence is the height of the tallest bar above the floor of an even split. It does not look at the other bars, which is why a close race and a lopsided one can get the same number.

2.3 Slide 7: Score: ordered levels, an expected level, and a spread

The score is the probability-weighted level, so it can sit between levels; confidence is how tight the mass is.

What the slide shows The vendor's bug-severity example: three levels (0 cosmetic, 1 degraded with a workaround, 2 blocking) with probabilities 0.0, 0.57 and 0.43; the score $0.57 + 2 \cdot 0.43 = 1.43$; the confidence formula with its evaluation to 0.35; and the contrast with the Choice formula on the pair $(0, 0.5, 0.5)$ and $(0.5, 0, 0.5)$. A panel on using scores: 2–10 levels numbered from 0, a score is an expectation and not a label, the same construction in G-Eval, composite scoring, and thresholding on the score.

Details A Score question's criteria are an ordered list of 2–10 levels, numbered from 0 in the order given, each a string or an object with a description and signals or examples[†]. The response has three parts: `probabilities`, one per level, summing to 1; `score`, the probability-weighted level; and `confidence`. The score is

$$\text{score} = \sum_{i=0}^{n-1} i\, p_i \qquad \text{(“multiply each level number by its probability, then add the results”}^{\dagger}\text{)}.$$

For the example, $0 \cdot 0 + 1 \cdot 0.57 + 2 \cdot 0.43 = 1.43$: the mode is level 1, but the score reflects the pull toward level 2. The vendor's confidence for a Score is

$$\text{confidence} = \max\left(0,\ 1 - \frac{\sum_i p_i\, |i - m|}{\text{MAD}_{\text{unif}}}\right), \qquad m = \text{most likely level}, \qquad \text{MAD}_{\text{unif}} = \tfrac{1}{n} \textstyle\sum_{i=0}^{n-1} \left| i - \frac{n-1}{2} \right|.$$

The numerator is the probability-weighted distance, in levels, from the mode; the denominator is the mean absolute deviation of a uniform distribution over the levels, so the ratio is the spread relative to the spread of knowing nothing. For $n = 3$, $\text{MAD}_{\text{unif}} = \frac{1}{3}(1 + 0 + 1) = \frac{2}{3}$; the example's numerator is $0 \cdot 1 + 0.57 \cdot 0 + 0.43 \cdot 1 = 0.43$; confidence $= 1 - 0.43/0.667 = 0.355$, reported as 0.35 (derived from the docs' formulas; the docs report the same number). The two three-level examples show why the Score formula is distance-aware: $(0, 0.5, 0.5)$ has mode 1 or 2 (a tie; either gives the same answer), numerator 0.5, confidence $1 - 0.75 = 0.25$; $(0.5, 0, 0.5)$ has numerator $0.5 \cdot 2 = 1.0$, so $1 - 1.5 < 0$ and the confidence is floored at 0. The Choice formula would give 0.25 to both, because it only sees $p_{\max}$. Mass two levels apart is a real disagreement about severity; mass on neighbouring levels is the ordinary fuzziness of a rubric.

The panel's practical points. A score of 1.43 is an expectation; round it only when a label is needed, and keep it when summing, ranking or thresholding (the guardrail cookbook turns a review into a block at severity ≥ 2.0 on a 0–3 scale[†]). The same construction appears in G-Eval (Liu et al. 2023), which replaced the argmax of an LLM judge's rating tokens with their probability-weighted sum to get finer, less tied scores; ordinal regression is the classical name. Composite scoring is the vendor's pattern for broad judgments: ask several atomic Score questions and combine them with weights in code (the docs' examples use weights such as 0.4/0.4/0.2), so that the weights are yours to audit.

Terms

level One of the 2–10 ordered positions of a Score question, numbered from 0 in the order the criteria list them.

legend The map from level number to level description echoed in a Score response.

expected level The probability-weighted level $\sum_i i\,p_i$; the `score` field.

mode The most likely level, $m = \arg\max_i p_i$; the anchor of the Score confidence formula.

mean absolute deviation The average distance of a distribution's mass from a reference point; $\mathrm{MAD}_{\mathrm{unif}}$ is that of the uniform distribution from the middle level.

composite scoring Asking several atomic Score questions and combining them with weights set in code; the vendor's pattern for broad quality judgments.

probability-weighted score G-Eval's name for the expected level computed from an LLM judge's rating-token probabilities.

Intuition

A Score is a centre of mass on a ruler, and its confidence is how spread out the mass is. Two lumps at opposite ends have a centre in the middle and no confidence at all, which is exactly right.

2.4 Slide 8: Noul and Box: a single probability, and four edges

Noul is the workhorse for verification and detection; Box is the first step outside text, and still a preview.

What the slide shows On the left, a Noul drawn as a 0–1 scale with an "unsure" band around 0.5 and a marker at $p = 0.87$; a panel on reading a Noul (no separate confidence, define both sides, two thresholds). On the right, a Box drawn as a rectangle inside an image with its four edges labelled from the docs' mug example (top 0.25, bottom 0.82, left 0.44, right 0.84, confidence 0.92), and a panel on using Box in its preview state.

Details A Noul returns one number, the probability that the answer is yes, and the docs deliberately return no separate confidence: "the number is the answer and the certainty in one"†. A value near 0.5 is the model saying it cannot tell; a value near 0 or 1 is a confident no or yes. If a 0–1 certainty is wanted, $|2p - 1|$ is the docs' suggestion, which is 0 at $p = 0.5$ and 1 at either end, and which coincides with the Choice confidence formula for two options (slide 6). Two design rules follow from the shape. First, the criteria should define both the yes case and the no case, with examples and a `not_for` boundary; otherwise the model decides where the boundary is, and it will not be where you assumed. Second, use two thresholds rather than one: the guardrail cookbook sends a message to review when a hazard's probability reaches 0.35 and takes the hazard's action (block, support, review) at 0.70 in its strict profile or 0.85 in its permissive one†; the band between the two is the human's. A structural caveat noted by independent testers†: a Noul and its negation asked as two questions need not sum to 1, because each question is evaluated on its own (slide 24).

Box is the first primitive outside text and the least settled†. The answer is a `box` with `top`, `bottom`, `left`, `right`, each a fraction of the image from the top-left corner, with `left`<`right` and `top`<`bottom`, plus a confidence. It needs Jev 1.14 (`jev-preview` or `jev-1.14.0`) and the images header. It takes no criteria, needs an image in the request, and always returns a box even when the target is absent, so presence must be checked separately with a Noul; one target per question; targets a few dozen pixels wide in a large image can be missed, so crop first. The image limits apply: PNG or JPEG, 280–1,568 pixels per side, 16 MiB per file, listed once in the request's `assets` map and referenced by key from the state or the question. The vendor's framing is explicit that images and Box "aren't yet up to our standards to call ready for production", and the deck treats them as a direction rather than a tool.

Terms

unsure band A range of Noul probabilities around 0.5 (0.4–0.6 in the docs' spam example) treated as "cannot tell" and routed to review.

Noul certainty $|2p - 1|$, the distance of a Noul probability from 0.5 rescaled to 0–1; the docs' suggested certainty for a Noul, equal to the two-option Choice confidence.

review threshold / action threshold The two thresholds of the guardrail cookbook: at the lower one a case goes to a person, at the higher one the configured action fires.

assets The top-level map in a request that lists images once by key, as base64 or multipart, for reference from state or questions.

presence check A Noul asked alongside a Box to establish that the target exists, since a Box always returns a box.

Intuition

A Noul is a dial, not a switch: 0.5 means "I cannot tell", and the two thresholds you set around it are where the application decides what "cannot tell" costs. A Box is a Noul's younger sibling who still needs supervision.

2.5 Slide 9: State and questions: facts go in, judgments are asked

One request evaluates one state against many independent questions; batching costs nothing in accuracy.

What the slide shows A request body with a structured state (a ticket, an order with two charges of 49, and a refund policy) and two questions (a Choice on the owning team and a Noul on whether a refund is due under the policy). A panel of rules from the docs. On the right, two stat tiles from the parallel-questions cookbook (12.2× cheaper, 10.0× faster for 13 questions in one call rather than 13 calls), a note on the experiment, and a panel of request rules: context limits, body size, nesting, the long-context header, and a derived cost example.

Details The docs' guidance on state is simple and consistent[†]. State may be a string, an object or an array; use an object for most requests so that each part has a name and questions can refer to it with a backticked path (`` `order` ``, `` `refund_policy` ``). Keep content and facts in the state and the judgments in the questions: the refund request and the policy go in the state, and the question asks whether the request is valid under the policy. Include only the context the current questions need, because irrelevant state measurably hurts accuracy ("context rot" on the jaggedness page; slide 24). Keep deterministic logic in code: days overdue, amounts, counts and dates are computed by the application and passed in as fields, not asked of the model. Questions should be short, atomic and explicit; "broad questions hide several judgments behind one answer". All questions in a request see the same state and are evaluated independently, so one answer cannot influence another, and the three types may be mixed freely.

Batching is the economic consequence of independence. The parallel-questions cookbook[†] sends the pinned Wikipedia article on the GDPR (53,777 characters) with 13 questions (8 Noul, 2 Choice, 3 Score) in one call, and the same 13 questions one per call, five runs each, on `jev-1.12`. The answers are the same: eleven of thirteen had a standard deviation of exactly zero under both strategies, and the two with run-to-run noise had the same noise either way. The cost was $0.000497 for the single call against $0.00609 for thirteen, and the time 0.27 s against 2.71 s summed, which the cookbook reports as 12.2× cheaper and 10.0× faster. The cost ratio is nearly the question count because the document is sent once instead of thirteen times, and output tokens are free; the time ratio assumes the thirteen calls run sequentially. Speculative fan-out (slide 20) is the same observation turned into a pattern.

The request rules[†] bound what one call can contain. Jev 1.13 and 1.14 accept 64k tokens per request and 32k per question (the state plus the longest question); the body may be at most 32 MiB, compressed with gzip or zstd if desired; JSON nesting is limited to 64 levels in each of the state, instructions and criteria; numbers must be finite, so large identifiers travel as strings. The header `TypeSafe-Danger-Zone-Long-Context: true` raises both limits to 262k tokens at the same price, with the vendor's own warning that it is "slower and less accurate". The derived cost example: a 10k-token state with 20 questions costs its input, roughly 10k tokens (the questions are short) at $0.042 per million, about $0.0004.

Terms

backticked path A reference in a question's instructions to a field of the state, such as `` `order` `` or `` `ticket.messages` ``.

atomic question A question that asks for one judgment; the opposite of a broad question that bundles several.

independent evaluation The property that each question in a request is answered without reference to the others; enables batching and forbids cross-question reasoning.

speculative fan-out Asking every question that might be needed in one call and letting code decide afterwards which answers matter. Slide 20.

context limit The maximum tokens per request (64k) and per question (32k) on Jev 1.13/1.14; 262k behind the long-context header.

context rot The vendor's term for the accuracy loss caused by irrelevant material in the state. Slide 24.

alias A model name that points to a version and moves: `jev-latest` (the stable release) and `jev-preview` (the newest release). Pin versions in production.

Intuition

The state is the exhibit and the questions are the jury's instructions. Send the exhibit once, hand the jury every instruction at the same time, and keep the arithmetic for the clerk.

3 Calibration (slides 10–13)

3.1 Slide 10: Calibration: the probability has to mean something

Of all the answers given with probability p, about a fraction p should be right: a population property.

What the slide shows A reliability diagram, labelled illustrative: five bins of predicted probability on the horizontal axis, the fraction of correct answers in each bin as a bar, and the diagonal that a calibrated model would follow; the drawn model sits below the diagonal (overconfident). A panel defines three numbers (expected calibration error, Brier score, log loss) and a note says what a decision model promises.

Details *Calibration* is a statement about a population of predictions, never about one. Take every answer a model gave with probability near p; if about a fraction p of them turned out right, the model is calibrated at p. The vendor states it the same way[†]: of the answers given 0.2, about 20% should come true; of those given 0.8, about 80%; "these rates describe groups of predictions, not any single answer". The reliability diagram (DeGroot and Fienberg 1983) is the picture: bin the predictions by probability, plot the empirical accuracy of each bin against its mean predicted probability, and compare with the diagonal. Below the diagonal is overconfidence (the model says 0.9 and is right 70% of the time), above it underconfidence.

Three numbers summarize the diagram.

$$\text{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left|\text{acc}_b - \text{conf}_b\right| \qquad \text{(Guo et al. 2017; } B \text{ bins, } n_b \text{ predictions in bin } b\text{).}$$

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} \left(p_{ik} - \mathbb{1}[y_i = k]\right)^2 \qquad \text{(Brier 1950; multiclass form).}$$

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^{N} \log p_i(y_i) \qquad \text{(cross-entropy against the label).}$$

ECE is the weighted average gap between the bars and the diagonal; it is cheap and standard and it depends on the binning (slide 26). The Brier score is the squared error of the whole probability vector against the one-hot label, and Murphy's decomposition splits it into three parts: *reliability* (how far the bins sit from the diagonal; calibration error), *resolution* (how much the bins' accuracies differ from the overall base rate; how informative the probabilities are) and *uncertainty* (the base rate's own variance, which no model can change): Brier = reliability − resolution + uncertainty. The decomposition is why calibration alone is not enough: a model that always predicts the base rate is perfectly calibrated and useless (zero reliability error, zero resolution), and a model that is sharp but wrong has high resolution and high reliability error. Log loss is the cross-entropy a classifier is trained on; it is unbounded when a wrong answer gets probability near zero, which is a feature during training and a nuisance in evaluation.

What a decision model promises is that its returned `probabilities` sit on the diagonal for your questions. The vendor can only have checked this on its own distribution of questions; slides 26–27 are about how far it transfers, and the honest summary is "it depends on the task, from an ECE of 0.02 to 0.25".

Terms

reliability diagram A plot of empirical accuracy against predicted probability, by bin; calibrated predictions lie on the diagonal (DeGroot and Fienberg 1983).

expected calibration error (ECE) The bin-weighted mean absolute gap between accuracy and predicted probability; the standard scalar for calibration (Guo et al. 2017).

Brier score The mean squared error between the probability vector and the one-hot label; a strictly proper scoring rule (Brier 1950).

log loss $-\frac{1}{N}\sum \log p_i(y_i)$, the negative log-likelihood of the labels; cross-entropy; strictly proper and unbounded.

Murphy decomposition Brier = reliability − resolution + uncertainty: calibration error, informativeness, and the irreducible base-rate variance.

overconfidence / underconfidence Predicted probabilities larger / smaller than the empirical accuracy at that probability.

sharpness How concentrated a model's predictions are, independent of their correctness; the goal is to maximize sharpness subject to calibration (Gneiting).

base rate The overall frequency of a class; predicting it always is calibrated and uninformative.

Intuition

Calibration is a promise about the long run: "when I say 80%, I am right four times in five." It is checked with a histogram, not with a single answer, and it is compatible with being useless, so it is necessary and not sufficient.

3.2 Slide 11: Proper scoring rules make honesty the best policy

Score the probability, not the guess: a strictly proper rule is optimized only by reporting the true belief.

What the slide shows A plot of expected loss against the reported probability q of "yes" when the true probability is $p = 0.7$: the Brier curve $p(1-q)^2 + (1-p)q^2$ and the log-loss curve $-p\ln q - (1-p)\ln(1-q)$ both bottom out at $q = 0.7$; the 0/1 loss of the argmax is flat on each side of $q = 0.5$. A panel explains why this matters for decision models: Brier and log loss are strictly proper, accuracy is not, and preference rewards are not either.

Details Suppose a forecaster believes the probability of "yes" is p and is asked to report a number q, knowing that a loss $S(q,y)$ will be charged once the outcome y is known. The expected loss is $pS(q,1) + (1-p)S(q,0)$. A scoring rule is *proper* if this expectation is minimized at $q = p$ and *strictly proper* if $q = p$ is the unique minimizer (Gneiting and Raftery 2007). For the Brier score,

$$\mathbb{E}[\text{Brier}] = p(1-q)^2 + (1-p)q^2, \qquad \frac{d}{dq} = -2p(1-q) + 2(1-p)q = 2(q-p), \qquad \text{zero only at } q = p.$$

For log loss, $\mathbb{E} = -p\ln q - (1-p)\ln(1-q)$, whose derivative $-p/q + (1-p)/(1-q)$ also vanishes only at $q = p$. Both curves on the slide are derived from these expressions with $p = 0.7$. The 0/1 loss of the argmax is different: the forecaster is charged 1 if the argmax is wrong, so any $q > 0.5$ yields expected loss $1 - p = 0.3$ and any $q < 0.5$ yields $p = 0.7$; the curve is flat on each side, and the forecaster has no reason to report 0.7 rather than 0.99.

The three consequences for decision models. First, a classifier trained by cross-entropy is trained on a strictly proper rule, which is why classifiers can be calibrated at all and why Niculescu-Mizil and Caruana found logistic regression calibrated out of the box. If the vendor's RLCD rewards a proper scoring rule on the returned distribution, the calibrated answer is the reward-maximizing answer, and slide 14 argues that this is the only reasonable reading of "reinforcement learning for calibrated decisions". Second, accuracy-only evaluation rewards guessing. Kalai, Nachum, Vempala and Zhang (2025) make this the explanation of hallucination: benchmarks give zero for "I don't know" and sometimes full credit for a confident bluff, so a model optimized against them learns to bluff; a decision model that is scored on its probability, and an application that treats 0.5 as "cannot tell", remove the incentive. Third, preference rewards are not proper either. Leng et al. (2024) show that RLHF reward models prefer responses that state high confidence regardless of correctness, so a policy optimized against them learns to sound sure; this is the mechanism behind the calibration collapse of slide 12 and the reason a decision model needs a different objective.

Terms

scoring rule A loss $S(q,y)$ charged to a reported probability q once the outcome y is known.

proper / strictly proper A scoring rule whose expected loss is minimized at the true belief / uniquely minimized there (Gneiting and Raftery 2007).

0/1 loss The loss that charges 1 for a wrong argmax and 0 otherwise; accuracy's complement; not proper.

cross-entropy training Minimizing log loss over labelled examples; the standard classifier objective

and a strictly proper rule.

preference reward A reward model's score of a response, trained from pairwise human preferences; rewards confident-sounding text and is not a proper scoring rule.

guessing incentive The property of accuracy-only evaluation that a confident guess scores at least as well as an honest abstention; Kalai et al. (2025).

Intuition

A proper scoring rule is a bet whose best strategy is to tell the truth. Accuracy is a bet where bluffing never costs extra, and models learn to bluff.

3.3 Slide 12: Do language models know what they know? Until post-training

Base models are calibrated in the right format; preference tuning breaks it; asking in words helps but overshoots.

What the slide shows Four panels of findings: deep networks drifted out of calibration (Guo et al. 2017; Minderer et al. 2021); base language models are calibrated when the format fits (Kadavath et al. 2022; Zhao et al. 2021); post-training breaks it (the GPT-4 report; Xiao et al. 2025); asking in words helps but overshoots (Tian et al. 2023; Xiong et al. 2023; Kapoor et al. 2024). A closing line: calibration is cheap to have and easy to lose, and none of nine recent model reports even report it (Sanz-Guerrero and von der Wense 2026).

Details The deep-learning finding came first. Guo et al. (2017) compared a 1998 LeNet with a 2016 ResNet on image classification and found the modern network far more overconfident, with depth, width and weight decay all pushing in the same direction and negative log-likelihood overfitting even as accuracy improved; one temperature fitted on held-out NLL repaired most of the gap. Minderer et al. (2021) revisited the question with newer architectures and found that architecture is a major determinant of calibration, with some recent non-convolutional models among the best calibrated, so calibration is not simply a function of size or accuracy.

For language models the finding was more favourable than expected. Kadavath et al. (2022) showed that large models are well calibrated on multiple-choice and true/false questions "in the right format" (options presented jointly, answered by letter), that they can predict the probability that their own sampled answers are true ($P(\text{True})$), and that calibration degrades when the format shifts. Zhao et al. (2021) found a different distortion: a content-free input ("N/A") exposes a label prior, a bias toward certain answers caused by the prompt's examples and order, and correcting it ("contextual calibration") improves few-shot accuracy by up to 30 points and cuts variance across prompts.

Then post-training broke it. The GPT-4 technical report (2023), Figure 8, shows the pre-trained model's reliability diagram on an MMLU subset close to the diagonal with an ECE of 0.007 and the PPO-tuned model's far from it with an ECE of 0.074, with the caption "the post-training hurts calibration significantly". Zhu et al. (2023) found the same across open models: pretraining improves calibration with scale, instruction tuning worsens it, and synthetic instruction data worsens it more. Xiao et al. (2025) named the mechanism *preference collapse*, the probability mass piling onto the preferred answer, and showed a calibration-aware fine-tuning objective that brings ECE from 14–20% back to 2.4–6.5%.

Asking the model in words is a partial remedy. Tian et al. (2023) found that for RLHF models a verbalized confidence ("I am 80% sure") is better calibrated than the conditional token probability, with gpt-3.5's ECE on TriviaQA falling from 0.140 to 0.050; Xiong et al. (2023) evaluated prompting, sampling and aggregation strategies and found models still overconfident, with stated numbers clustered at a few values; Kapoor et al. (2024) argue that prompting is not enough and that about a thousand graded examples with LoRA on internal

features is what produces uncertainty estimates that generalize. The closing line is the lesson a decision model is built on: calibration survives when the model is trained and scored in the exact format it answers in, and it does not survive a change of format or a round of preference tuning; and Sanz-Guerrero and von der Wense (2026) note that none of nine recent model reports or cards report calibration at all.

Terms

P(True) Kadavath et al.'s probe: the model's own probability that a sampled answer of its own is correct.

label prior / contextual calibration A bias toward certain labels induced by the prompt; estimated with a content-free input and divided out (Zhao et al. 2021).

preference collapse The concentration of probability onto the preferred answer caused by preference tuning (Xiao et al. 2025).

verbalized confidence A probability the model states in words rather than one read from its logits (Lin et al. 2022; Tian et al. 2023).

calibration-aware fine-tuning A fine-tuning objective that penalizes miscalibration while preserving alignment (Xiao et al. 2025).

format shift Asking a model in a format different from the one it was calibrated in; degrades calibration (Kadavath et al. 2022).

Intuition

A base model is an honest statistician who is then sent to charm school. It comes back more pleasant and more sure of itself. A decision model is the statistician kept out of charm school and asked only the questions it was trained to answer.

3.4 Slide 13: Four ways to get a probability out of a language model

Read the logits, ask in words, sample and count, or train a head. A decision model is the fourth, sold as a service.

What the slide shows A four-row table: option logprobs (MMLU style), verbalized confidence, sampling consistency, and a trained head or decision model; for each, what it needs, what speaks for it and what speaks against it. A closing line on why the fourth wins on cost.

Details *Option logprobs.* Present the options with letters, ask the model to answer with one, and read the log-probability of each letter at the answer position; normalize over the letters. It needs logits or a `logprobs` API and costs one forward pass (a prefill, no decode), which makes it the cheapest route and the classical classifier readout. It can be repaired with a temperature (slide 16) or a label-prior correction (Zhao et al. 2021). Against it: token bias on the letters themselves (Zheng et al. 2023 show that selection bias is mostly a prior over the option identifiers, and remove it with PriDe); sensitivity to option order (Pezeshkpour and Hruschka 2023: 13–75% performance swings from reordering); and, for instruction-tuned models, the fact that the first-token probability disagrees with the text the model would actually generate at rates reaching over 60% (Wang et al. 2024, "My answer is C"), so the readout and the model's "real" answer can diverge.

Verbalized confidence. Ask the model how sure it is. It works on closed models through any chat API and the number is human-readable; for RLHF models it is the better-calibrated option (Tian et al. 2023). Against it: overconfidence and clustering (Xiong et al. 2023), sensitivity to the prompt, and the cost of generating the words.

Sampling consistency. Sample k answers and measure agreement. It works for free-form answers where there are no options to score, and agreement is a good difficulty signal for cascades (Yue et al. 2023 route on

it). Against it: it measures stability, not truth (a model can be consistently wrong), and it costs k calls per decision.

A trained head or a decision model. Train a model, or a head on a frozen model, to output the distribution directly, with a proper scoring rule as the loss, in the exact format it will be asked in. One pass, batched questions, no text. Against it: fixed question shapes, no reasoning, and calibration certified on someone else's data when the model is a vendor's (slide 27). The cost argument is the closing line: verbalizing and sampling pay for generated tokens; a decision head emits a few numbers per question and nothing else, which is what makes "output tokens are free" and 100 ms plausible. Whether it also wins on quality is the empirical question of slide 27, where a 7B logprob readout with a temperature lands within a few points on one task.

Terms

logprobs API An API option that returns the log-probabilities of the top tokens at each position; available on some generative APIs, absent on others.

token bias / selection bias A prior over option identifiers (A, B, C, D) independent of content; the main cause of order sensitivity in multiple-choice readouts (Zheng et al. 2023).

PriDe Zheng et al.'s label-free debiasing that estimates the identifier prior from permutations and divides it out.

sampling consistency The agreement among k sampled answers; self-consistency when used to pick the answer, a difficulty signal when used to route.

first-token mismatch The disagreement between the probability of the first answer token and the text answer an instruction-tuned model generates (Wang et al. 2024).

decision head A small network on top of a frozen model's hidden states that produces the option probabilities directly. Slide 23.

Intuition

Four ways to make a chatbot vote: peek at its ballot, ask it how sure it is, make it vote ten times, or train a voter. The fourth is cheapest because it never has to talk.

4 Training (slides 14–16)

4.1 Slide 14: What is known about how a decision model is trained

The vendor gives a name and a goal; the published recipes that match the goal all reward a proper scoring rule.

What the slide shows Two panels. Left, what TypeSafe says: RLCD, "reinforcement learning for calibrated decisions", trains the model to return decisions and calibrated probabilities instead of generated text; nothing else is public; treat it as a claim. Right, the nearest published recipes: Lin, Hilton and Evans (2022); Rewarding Doubt (Bani-Harouni et al. 2025); RLCR (Damani et al. 2025); SaySelf, LACIE and Leng et al. (2024). A closing line on what this implies, marked as inference.

Details The public record on Jev's training is one paragraph of documentation and a few interview sentences[†]. The documentation names RLCD, expands it as "reinforcement learning for calibrated decisions", and says that it trains the model "to return decisions and calibrated probabilities instead of generated text". It defines calibration as the population property of slide 10. In interviews the CEO has said that the training data is entirely synthetic, that a paper is "not yet" coming, and that he will not discuss the architecture; the vendor

has described the model as transformer-based with "a new architecture, a new sampler, and a new training algorithm" and a "parallel sampler" that produces all answers in one query rather than autoregressively. There is no paper, no model card, no weight release, no parameter count, and no per-customer fine-tuning. One warning about the name: RLCD is also the title of Yang, Klein, Celikyilmaz, Peng and Tian (2023), "Reinforcement Learning from Contrastive Distillation for Language Model Alignment", which generates preference pairs from positive and negative prompts and has nothing to do with calibration; searches for the acronym return it first. The first independent at-scale study (Rafe and Das 2026) explicitly treats RLCD as an unverified vendor claim, and so does this lecture.

What the published literature offers is a set of recipes that all share one feature: the reward is a proper scoring rule on a stated or returned probability. Lin, Hilton and Evans (2022) fine-tuned GPT-3 to state a verbal probability alongside its answer and found the stated probabilities calibrated, and still moderately calibrated under distribution shift. Bani-Harouni et al. (2025, "Rewarding Doubt") use reinforcement learning with a reward that is a logarithmic scoring rule on the stated confidence, and prove that the optimal policy under that reward is exactly calibrated. Damani et al. (2025, RLCR) add a Brier score on the verbalized confidence to the usual correctness reward of RLVR, and find that "while ordinary RL hurts calibration, RLCR improves it" with no loss of accuracy; this is the closest published analogue to a method that rewards calibrated decisions. SaySelf (Xu et al. 2024) uses PPO with a quadratic confidence reward after supervised training on self-reflective rationales; LACIE (Stengel-Eskin et al. 2024) optimizes preferences against a simulated listener; Leng et al. (2024) repair the RLHF reward model's taste for confident text.

The closing inference is stated as such. A decision model's output is a distribution over supplied options, so the natural reward is a proper scoring rule on that distribution against a label or an outcome. Supervised cross-entropy would do the same for a fixed label set; the plausible reasons to use RL instead are that rewards can come from outcomes (was the routed case resolved, did the extraction ship without correction) rather than from labels, that one policy must handle every question shape and every option set, and that calibration can be scored over batches rather than per example. None of this is disclosed, and the deck says so.

Terms

synthetic training data Data generated by models rather than collected from people; the vendor states Jev's training data is entirely synthetic[†].

parallel sampler The vendor's term for producing every answer of a request in one pass rather than token by token[†]; consistent with a readout head, undisclosed.

Rewarding Doubt Bani-Harouni et al. (2025): RL with a log-scoring-rule reward on stated confidence; the optimal policy is calibrated.

RLCR Damani et al. (2025): reinforcement learning with calibration rewards, correctness plus Brier on the verbalized confidence.

SaySelf Xu et al. (2024): supervised fine-tuning on self-reflective rationales followed by PPO with a quadratic confidence reward.

LACIE Stengel-Eskin, Hase and Bansal (2024): listener-aware fine-tuning that optimizes calibration as a preference against a simulated listener.

contrastive distillation (RLCD of Yang et al.) An unrelated alignment method that shares the acronym; preference pairs generated from contrasting prompts.

Intuition

If you want a model to report its true belief, pay it with a proper scoring rule. Every public recipe that works does exactly that; the vendor's name for its method says it wants the same thing and shows nothing of how.

4.2 Slide 15: RLHF, RLVR, RLCD: three objectives, three kinds of model

What the signal is, what the policy emits, what gets rewarded, and what each one is known to get wrong.

What the slide shows A table with four rows (RLHF, RLVR, RLCD, and RLCR as the academic hybrid) and four columns: the training signal, what the policy emits, what is rewarded, and the known failure. A closing line: inference follows the objective, and the three are complements.

Details *RLHF* (InstructGPT, Ouyang et al. 2022) trains a policy that emits free text against a reward model fitted to human (or AI) preferences. It is rewarded for what raters prefer. Its known failures are sycophancy and confident-sounding errors, and, for the purposes of this lecture, the collapse of the policy's probabilities onto the preferred answer (slide 12). Its inference profile is full generation: it pays per output token and takes seconds.

RLVR, reinforcement learning from verifiable rewards (the name popularized by Tülu 3 and DeepSeek-R1 in 2024–25), trains a policy that emits a chain of thought followed by an answer against a reward that checks the answer: a unit test, a final number, a string match. It is rewarded for the right final answer. Its known failures are long reasoning on easy cases, overconfidence on wrong answers (Mei et al. 2025 find reasoning models over 85% confident on wrong answers, with deeper reasoning making it worse, against Yoon et al. 2025 who find reasoning models better calibrated in most settings; the two disagree about which tasks and which models), a "reasoning tax" on knowing one's limits (Zeng et al. 2025), and a degradation of abstention of about 24% on average when models are fine-tuned for reasoning (AbstentionBench, Kirichenko et al. 2025). Its inference is the longest of the three.

RLCD† trains, on the vendor's description, a policy that emits a distribution over the supplied options and no text, against labelled or verifiable decisions scored by what the deck infers to be a proper rule. It is rewarded for a calibrated probability, not only for the right argmax. Its known failures are undisclosed by the vendor beyond the jaggedness list (slide 24), and documented by third parties as flips under injected context (slide 25). Its inference is one forward pass per request.

RLCR (Damani et al. 2025) is the academic hybrid: RLVR's verifiable reward plus a Brier score on the stated confidence, so the policy still reasons and generates but is rewarded for calibration as well as correctness. It inherits RLVR's inference cost.

The closing line is about complementarity rather than competition. The inference profile of each kind of model follows from what it emits: text costs tokens and time, a distribution costs neither. The design that uses all three is a reasoning model for the cases a decision model flags as hard, and a decision model for the checks a reasoning model is too slow and expensive to run on everything (Section 5).

Terms

RLHF Reinforcement learning from human feedback: policy optimization against a preference-trained reward model (Ouyang et al. 2022).

RLVR Reinforcement learning from verifiable rewards: policy optimization against programmatic checks of the final answer; the training of reasoning models.

chain of thought The reasoning text a model emits before its answer; the output RLVR optimizes and a decision model does not produce.

sycophancy Telling the rater what the rater wants to hear; a known failure of preference optimization.

reasoning tax Zeng et al.'s term for the loss of knowledge-boundary awareness that accompanies reasoning training.

AbstentionBench Kirichenko et al. (2025): twenty datasets of unanswerable questions; reasoning fine-tuning degrades abstention by about 24%.

Intuition

Three objectives, three products: RLHF makes a pleasant writer, RLVR a careful solver, RLCD a fast judge. You want the judge to decide which cases deserve the solver.

4.3 Slide 16: Calibration after the fact: temperature, Platt, isotonic

One scalar fitted on a few hundred labelled examples repairs most miscalibration without touching the argmax.

What the slide shows Three bar groups showing the softmax of the logits $(2.0, 1.0, 0.2)$ at $T = 0.5$ (0.86, 0.12, 0.02), $T = 1$ (0.65, 0.24, 0.11) and $T = 2$ (0.50, 0.30, 0.20); the temperature-scaling formula and its fitting rule; one-line descriptions of Platt scaling, isotonic regression and Dirichlet calibration. A panel on what post-hoc calibration does and does not do, ending with evidence that it applies to decision models too.

Details Temperature scaling (Guo et al. 2017) divides the logits by a scalar before the softmax:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \qquad T \text{ chosen to minimize the log loss on a held-out labelled set.}$$

$T > 1$ flattens the distribution (the $T = 2$ bars), $T < 1$ sharpens it (the $T = 0.5$ bars), and no value of T changes the argmax or the ranking of options, which is why it fixes calibration and nothing else. The bars are derived: at $T = 1$, $e^2 = 7.39$, $e^1 = 2.72$, $e^{0.2} = 1.22$, sum 11.33, probabilities 0.652, 0.240, 0.108; at $T = 2$ the exponents halve; at $T = 0.5$ they double. Platt scaling (1999) fits a sigmoid $\sigma(az + b)$ to a binary classifier's score, of which temperature scaling is the one-parameter special case ($b = 0$). Isotonic regression (Zadrozny and Elkan 2002) fits a monotone step function from score to probability, more flexible and more data-hungry. Dirichlet calibration (Kull et al. 2019) fits a linear map on log-probabilities for the multiclass case, which matters for Choice questions with many options, and introduces class-wise ECE.

The panel's four points. It needs labels from your own distribution: a few hundred to a few thousand graded answers suffice for a one- or two-parameter fit, which rarely overfits. It leaves ranking and accuracy alone: it moves the reliability diagram onto the diagonal and cannot make a bad selector good (slide 17). It breaks under shift: a temperature fitted on one domain is wrong on another (Ovadia et al. 2019), and Kumar, Liang and Ma (2019) show that scaling methods look better calibrated than they are when measured with binned ECE, and recommend binning after scaling to get a verifiable number. And it applies to decision models too. Rafe and Das (2026) found that Jev's raw probabilities overstated the prevalence of the coded attributes in 195,857 crash narratives and that recalibrating on 2,416 human labels cut the error by a factor of 3.3; Dibia (2026)† took a stock 7B model's option logprobs from an ECE of 0.10 to 0.03 with one fitted temperature on a prompt-injection task. The practical rule: whoever trained the model, calibrate on your own labels before you set a threshold.

Terms

post-hoc calibration Any method that adjusts a trained model's probabilities using a held-out labelled set without retraining it.

temperature T The scalar dividing the logits in temperature scaling; fitted by minimizing held-out log loss.

Platt scaling A sigmoid $\sigma(az+b)$ fitted to a classifier's scores (Platt 1999).

isotonic regression A monotone, piecewise-constant map from score to probability fitted on labelled data (Zadrozny and Elkan 2002).

Dirichlet calibration A linear map on log-probabilities followed by a softmax, for multiclass calibration (Kull et al. 2019).

class-wise ECE ECE computed per class and averaged; sensitive to miscalibration of minority options (Kull et al. 2019).

calibration set The held-out labelled examples used to fit a post-hoc correction or a conformal quantile; must resemble the traffic the model will see.

Intuition

Temperature scaling is the volume knob on a model's confidence: it makes every answer louder or softer together and never changes which answer is loudest. You set it by listening to a few hundred of your own examples, and you reset it when the room changes.

5 Selection (slides 17–20)

5.1 Slide 17: Selective prediction: answer when sure, hand off the rest

Sort decisions by confidence and stop answering below a threshold; error against coverage is the trade-off.

What the slide shows Two risk–coverage curves, labelled illustrative: the error rate among the answered cases (selective risk) against the fraction of cases answered (coverage). A good selector's curve falls steeply as coverage is reduced; a poor selector's hardly falls. A dotted line marks the 20% error of answering everything; a marked point reads 60% coverage at 5.6% error. A panel gives the formalism, distinguishes ranking from calibration, and introduces deferral.

Details A selective classifier (El-Yaniv and Wiener 2010; Geifman and El-Yaniv 2017) is a pair (f,g): f makes the prediction and $g(x) \in \{0,1\}$ decides whether to make it at all. Two quantities describe it:

$$\text{coverage } \phi = \mathbb{E}[g(x)], \qquad \text{selective risk } R = \frac{\mathbb{E}[\ell(f(x),y)\,g(x)]}{\mathbb{E}[g(x)]},$$

the fraction of cases answered and the error rate among them. With a confidence score, $g = \mathbb{1}[\text{conf}(x) \geq \tau]$, and sweeping τ from 1 down to 0 traces the risk–coverage curve from zero coverage to full coverage. The area under that curve, AURC (Geifman, Uziel and El-Yaniv 2019), summarizes the selector in one number; the illustrative good curve is $0.2\,c^{2.5}$ and the poor one $0.2\,c^{1.2}$, both reaching 20% error at full coverage, so the difference between them is entirely in how well the confidence orders right and wrong answers. Chow's (1970) reject option is the ancestor of the idea; Geifman and El-Yaniv's (2017) result is the modern demonstration: on ImageNet a selective classifier guaranteed a 2% top-5 error at about 60% coverage.

The panel's central distinction is between ranking and calibration. Selection needs confidence to *order* right and wrong answers, which is measured by the AUROC of confidence against correctness (Hendrycks and Gimpel 2016 established maximum softmax probability as the baseline for this); it does not need the confidence to be a calibrated probability. The two properties are independent, and the independent evidence on Jev shows both faces: Guo et al. (2026) find a median AUROC of 0.886 across the 31 yes/no benchmarks of their 44 (a good ranker) with a median per-benchmark ECE of 0.168, against 0.074 expected under perfect calibration with those sample sizes, while the ECE pooled over all benchmarks is 0.047: calibrated on average, off task by task, because the base rates differ. Such a model is still useful behind a threshold, but the threshold must be tuned per task rather than read off as a probability (slide 18).

Deferral generalizes rejection. If the rejected cases go to a person or to a larger model, their error matters too, and the optimal rule depends on both error rates (Madras, Pitassi and Zemel 2018; Mozannar and Sontag 2020 reduce the problem to cost-sensitive learning with a consistent surrogate). Jitkrittum et al. (2023) show, for cascades, that deferring on the small model's confidence alone is optimal only when the large model's error is roughly constant across inputs (slide 20).

Terms

selective classifier A pair (f, g) of a predictor and a selector; answers only where $g = 1$ (El-Yaniv and Wiener 2010).

coverage The fraction of inputs the selector answers, $\mathbb{E}[g]$.

selective risk The error rate among the answered inputs.

risk–coverage curve Selective risk as a function of coverage as the threshold is swept.

AURC Area under the risk–coverage curve; lower is better (Geifman et al. 2019).

AUROC of confidence The area under the ROC curve of confidence as a detector of errors; measures ranking quality (Hendrycks and Gimpel 2016).

reject option Chow (1970): declining to classify when the posterior is too close to the decision boundary.

learning to defer Training the selector with the downstream decision maker's errors in view (Madras et al. 2018; Mozannar and Sontag 2020).

Intuition

A selective classifier is a student who skips the questions she is unsure of. Her mark on the questions she answers depends entirely on how well she knows which ones to skip, and that skill is separate from how honest her "80% sure" is.

5.2 Slide 18: Thresholds are a cost decision, not a model setting

A calibrated probability turns "how sure must we be?" into arithmetic on the cost of each kind of mistake.

What the slide shows The rule "act when $p \geq p^* = c_{\text{FP}}/(c_{\text{FP}} + c_{\text{FN}})$" (Elkan 2001) and a table of four illustrative decisions with their two costs and the threshold they imply: flag a message for a safety reviewer (1 vs 50, $p^* = 0.02$); auto-approve a \$49 refund (49 vs 5, 0.91); route a ticket to the billing queue (2 vs 2, 0.50); trigger an expensive re-extraction (1 vs 20, 0.05). A paragraph on reading the table. A panel with the vendor's bands and its advice, and a note on how to test.

Details Elkan's (2001) argument is the foundation of cost-sensitive learning: learn calibrated probabilities, then decide by comparing expected costs. For a binary action (act or not) with the cost of a false positive

c_{FP} (acting when you should not), the cost of a false negative c_{FN} (not acting when you should), and correct decisions costing zero, the expected cost of acting is $(1-p)\,c_{\mathrm{FP}}$ and of not acting is $p\,c_{\mathrm{FN}}$; act when the first is smaller:

$$(1-p)\,c_{\mathrm{FP}} \le p\,c_{\mathrm{FN}} \quad\Longleftrightarrow\quad p \ge p^* = \frac{c_{\mathrm{FP}}}{c_{\mathrm{FP}} + c_{\mathrm{FN}}}.$$

The table's rows are illustrative costs in whatever unit fits. Flagging a message for a safety reviewer costs one reviewer-minute if it was harmless and 50 units if a harmful message reaches a user unflagged, so $p^* = 1/51 \approx 0.02$: a safety Noul should fire at almost any signal, and the reviewer absorbs the false alarms. Auto-approving a $49 refund costs $49 if the refund was not due and perhaps $5 of support time and goodwill if a due refund is wrongly denied, so $p^* = 49/54 \approx 0.91$: approve automatically only when the model is very sure. Routing a ticket to the billing queue costs about the same either way (a re-route), so $p^* = 0.5$: just take the argmax. Triggering an expensive re-extraction costs 1 for an unnecessary model call if the field was fine and 20 if a wrong field ships because it was not triggered, so $p^* = 1/21 \approx 0.05$: re-extract on almost any doubt. (The cascade cookbook's own gate is set the other way round, firing when the probability that a field is *wrong* exceeds 0.7; the arithmetic says that is cost-optimal only if an unnecessary re-extraction costs more than about two wrong fields.) Vickers and Elkin (2006) turn the same arithmetic into a plot, net benefit against threshold probability, that shows the range of thresholds over which a predictor is worth using at all.

The vendor's advice[†] is consistent with this without the arithmetic. The confidence page suggests three bands (act automatically at high confidence, proceed with caution in the middle, do not act at low), a "0.5 confidence floor" for catching answers the model is unsure of, and a higher bar for destructive operations (above 0.9 in its example). The confidence-routing page's voice-banking example sends any intent below 0.6 to a support agent, shows a balance at 0.6 or above, approves a transfer automatically above 0.85 and asks the user to confirm between 0.6 and 0.85; the confidence page gives the same transfer example with a 0.9 bar and a 0.5 floor, so the two pages disagree with each other by a few hundredths, which is itself a lesson about where thresholds come from. The guardrail cookbook reviews at 0.35 and blocks at 0.70. All of it ends with "start with conservative thresholds, test with your own data, and adjust as you observe results", which is Elkan's rule with the costs left to the reader.

Two cautions. Calibration is what makes a threshold transferable: if the probabilities are calibrated, p^* computed from costs is correct for every question that uses the same action; if they are not, the thresholds still work as tuned numbers, but they must be re-tuned per question and per model version, and the vendor's own note that `jev-latest` moves one week after each release makes that a recurring job. And the right way to test is a reliability diagram per question type on a labelled sample of your own traffic, reading off where the error rate crosses what each action can tolerate (slide 26).

Terms

cost-sensitive learning Making decisions that minimize expected cost given a cost matrix and calibrated probabilities (Elkan 2001).

false positive / false negative Acting when one should not / not acting when one should; their costs c_{FP}, c_{FN} set the threshold.

optimal threshold p^* $c_{\mathrm{FP}}/(c_{\mathrm{FP}} + c_{\mathrm{FN}})$ when correct decisions cost zero; act at or above it.

net benefit Vickers and Elkin's (2006) quantity, benefit minus harm weighted by the threshold odds, plotted against the threshold to show where a model helps.

confidence bands The vendor's three ranges of confidence mapped to act, proceed with caution, and do not act[†].

confidence floor A threshold below which no answer is acted on regardless of the action; 0.5 or 0.6 in the vendor's examples.

Intuition

The model says how likely; the threshold says how much a mistake hurts. Keep the two apart: the first is the model's job, the second is yours, and it lives in a line of code you can read aloud to an auditor.

5.3 Slide 19: Conformal prediction: sets with a coverage guarantee

From any probability vector to a set that contains the right answer $1-\alpha$ of the time, with no model assumptions.

What the slide shows The split-conformal recipe in four lines (Angelopoulos and Bates 2021) and the interpretation of set size as uncertainty. On the right, three tickets at $\hat{q} = 0.88$ (keep options with $p \geq 0.12$): a confident ticket yields a set of size 1 (route it), two ambiguous ones yield sets of size 3 (ask a person). A panel on the literature and on the ten-line wrapper a decision model makes possible.

Details Split conformal prediction (Vovk, Gammerman and Shafer 2005; the tutorial is Angelopoulos and Bates 2021) turns any score into a set-valued predictor with a finite-sample guarantee. With n labelled calibration questions (x_i, y_i) and a model's probabilities $\hat{p}(\cdot \mid x)$:

1. $s_i = 1 - \hat{p}(y_i \mid x_i)$ (nonconformity: how little probability the true answer got).
2. $\hat{q} =$ the $\lceil (n+1)(1-\alpha) \rceil / n$ empirical quantile of $s_1, \ldots, s_n$.
3. $C(x) = \{ y : \ 1 - \hat{p}(y \mid x) \leq \hat{q} \} = \{ y : \ \hat{p}(y \mid x) \geq 1 - \hat{q} \}$.
4. $\Pr[y \in C(x)] \geq 1 - \alpha$ for a new (x, y) exchangeable with the calibration set.

The guarantee is *marginal* (averaged over questions, not conditional on any one) and *distribution-free* (it needs no assumption about the model or the data beyond exchangeability, which in practice means that tomorrow's traffic looks like the calibration set). It says nothing about which element of the set is right, only that the right one is inside with probability at least $1 - \alpha$. The slide's example is illustrative: $\hat{q} = 0.88$ keeps every option with probability at least 0.12, so the confident ticket $(0.91, 0.03, 0.04, 0.02)$ yields $\{\text{billing}\}$, while $(0.62, 0.21, 0.12, 0.05)$ and $(0.40, 0.38, 0.15, 0.07)$ yield sets of three. The size of the set is the uncertainty, and it suggests a policy directly: a singleton can be acted on; a larger set means ask; an empty set (which the plain recipe can produce when nothing clears the bar) means the model has seen nothing like this. Adaptive variants (APS, RAPS) include options by cumulative probability instead, which gives sets that track the model's own uncertainty more closely at the cost of larger average size.

The literature has applied this to language models in three ways. Kumar et al. (2023) wrap conformal sets around LLM option probabilities on multiple-choice questions and find that set size tracks accuracy and that coverage is approximately maintained under mild shifts of subject. Ren et al. (2023, KnowNo) give a robot planner multiple-choice next steps and have it ask a human exactly when $|C(x)| > 1$, which is a decision-model pattern in all but name. Mohri and Hashimoto (2024) apply the idea to factuality, backing off sub-claims of a generated text until the retained claims meet a correctness target; Quach et al. (2023) do the same for sets of sampled generations. With a decision model, the Choice probabilities are the scores and the wrapper is ten lines of code; the two caveats are that the guarantee is about coverage, not about the argmax, and that it holds only as long as the calibration set resembles the traffic.

Terms

conformal prediction A method that turns any scoring model into a set-valued predictor with a finite-sample coverage guarantee (Vovk et al. 2005).

nonconformity score A measure of how unusual a label is for an input under the model; here $1 - \hat{p}(y \mid x)$.

miscoverage level α The allowed probability that the set misses the true answer; coverage is $1 - \alpha$.

marginal coverage A guarantee that holds on average over inputs, not for each input separately.

exchangeability The assumption that the calibration examples and the new example are drawn in a way that makes their order irrelevant; satisfied by i.i.d. sampling, violated by distribution shift.

prediction set $C(x)$, the set of options returned; its size is the model's uncertainty on that input.

KnowNo Ren et al. (2023): conformal sets over a planner's multiple-choice next steps, asking a human whenever the set is not a singleton.

Intuition

Conformal prediction is a promise about the net, not the fish: "the right answer is in this set nine times in ten." When the set has one element, act; when it has three, that is the model telling you to ask.

5.4 Slide 20: Cascades and fan-out: the cheap model decides who pays

A small calibrated model in front of a large one, firing it only when flagged; and many questions asked in one pass.

What the slide shows The vendor's structured-extraction cascade as a diagram: a document goes to gpt-5.4-mini ($0.75 / $4.50 per million tokens) which extracts the fields; Jev asks Nouls per field ("is the extracted value unsupported by, or absent from, the source text?"); if any field's probability of being wrong exceeds 0.7 the document goes to gpt-5.5 ($5 / $30) for re-extraction, otherwise it is accepted. A note records that the cookbook's only quoted number is for the expensive model run alone (about $0.10 per extraction at "≈0.81 quality" on an undefined score) and that the cascade is reported as a curve. A panel on speculative fan-out, and a panel on the literature of cascades and routers.

Details The cascade is the pattern that turns a calibrated cheap model into money[†]. The cookbook's version has three stages: a cheap generative model does the structured extraction; a decision model (`jev-1.12` in the cookbook) checks each extracted field with Nouls asking whether the value is unsupported by or absent from the source; the expensive generative model is called only for documents where at least one field's probability of being wrong exceeds the gate (`FIRE_T` $= 0.7$). The cost is dominated by the cheap model and the verifier, with the expensive model's price paid only on the flagged fraction. The cookbook's only quoted number is for gpt-5.5 run alone, "≈0.81 quality for ≈$0.10/extraction" on a quality score it does not define (each extraction is "scored by TypeSafe"); the cascade's own cost and quality appear only as points on a chart with the summary that sweeping the gate "buys you most of the top model's quality at a fraction of its cost". The lecture reports this as a vendor curve, not a number. The design question is the one of slide 18: the gate is a threshold on the probability that a field is wrong, and the cost of a wrong field shipping against the cost of an extra model call says where it should sit.

The literature has studied each piece. FrugalGPT (Chen, Zaharia and Zou 2023) built cascades of generative models with a learned scorer deciding whether to accept an answer or pass it up, and matched GPT-4's quality at up to 98% lower cost, or improved accuracy by 4% at the same cost. Jitkrittum et al. (2023) asked when deferring on the small model's confidence alone is enough, and answered: only when the large model's error is roughly constant across inputs; when the large model is a specialist, or the data

are noisy or shifted, the deferral rule should be learned from both models' errors. RouteLLM (Ong et al. 2024) trained routers on human preference data to send each query to a strong or a weak model and cut cost by more than 2× at matched quality, with routers that transfer across model pairs; Yue et al. (2023) used answer consistency as the difficulty signal and reached GPT-4 quality at about 40% of the cost; Gupta et al. (2024) showed that sequence-probability deferral has a length bias and that learned deferral on quantile features does better. With a decision model as the gate, Wu and Lim (2026) report 95% task success with 72.7% fewer strong-model calls in an agent, and note that the gain is small where a cheap generative cascade already routes well.

Speculative fan-out† is batching (slide 9) turned into a pattern: ask, in one call, every question you might need about a state, including the speculative ones whose relevance depends on other answers, and let code decide afterwards which answers matter. One round trip replaces a chain of calls, and the batching cookbook's 12× cost advantage over sequential calls applies.

Terms

verified cascade A cascade in which a decision model checks the cheap model's output and the expensive model is called only when a check fails.

flag threshold The Noul probability above which a field is considered unsupported and the document is escalated; 0.7 in the cookbook.

FrugalGPT Chen et al. (2023): LLM cascades with a learned acceptance scorer; up to 98% cost reduction at GPT-4 quality.

deferral rule The decision of whether to pass a case to the next model in a cascade; confidence-based or learned (Jitkrittum et al. 2023).

RouteLLM Ong et al. (2024): routers trained on preference data between a strong and a weak model.

difficulty signal Any quantity used to predict whether the cheap model will be right: its confidence, sampling consistency, a learned router.

Intuition

A cascade is a hospital triage desk: the nurse with the clipboard sees everyone, and the surgeon sees the ones the clipboard flags. The clipboard is cheap because it only asks yes/no questions, and the whole hospital's cost depends on how well it asks them.

6 Landscape (slides 21–23)

6.1 Slide 21: Jev: the product that named the category

Five weeks old on the day of this lecture; a price, a latency and a request volume that explain the attention.

What the slide shows Four stat tiles: $0.042 per million input tokens (output free); 78–127 ms median per task (Jev 1.14 / 1.13 on the Artificial Analysis Decision Index); 64k / 32k tokens per request / per question (262k behind a header); 956M requests in one week on OpenRouter for Jev 1.13 alone. Two panels: the company and its claims, and what others measured (latency, cost, availability).

Details The company†. TypeSafe AI was founded in 2024 in San Francisco; its CEO, Diogo Almeida, spent about four years at OpenAI and is a co-author of the InstructGPT paper (Ouyang et al. 2022). Jev 1.13 shipped on September 15, 2026, with a $40M seed round led by DCVC; 1.14 followed on October 14; and on October 9 the company announced an $870M round at a $7.5B valuation led by a16z with Sequoia

and DCVC participating, about three and a half weeks after launch (TechCrunch reports the amount and valuation without naming the round). The vendor's technical statements are few: the launch release says Jev "can process hundreds of outputs in parallel from a single prompt"; secondary accounts describe it as transformer-based with a new architecture, sampler and training algorithm; latency is quoted as 70–500 ms end to end; the CEO confirmed in an interview that all of the training data is synthetic; there is no paper "yet". The vendor's own workflow evaluation, with four workflows written by its own team and reference labels that are themselves the averaged outputs of two frontier models, puts Jev at 67.8% against GPT-5.6 Sol's 74.1%, at $0.0004 against $0.0836 per case and 0.4 s against 23.3 s; the deck reports the cost ratio of about 1/200 as derived and the whole table as vendor-run with a labelling method that measures agreement with other models rather than truth.

What others measured†. The Artificial Analysis Decision Index, an independent leaderboard, reports medians of 127 ms per task for Jev 1.13 and 78 ms for 1.14 at $41.53 per million tasks. OpenRouter's Banking77 run at eight concurrent requests measured p50 / p95 / p99 latencies of 175 / 270 / 353 ms for Jev against 2,266 / 3,004 / 3,835 ms for Claude Opus 5, and costs of $0.11 against $2.42 per thousand classifications; Anaconda measured a median of 88 ms per answer but found that about one request in twenty took around 7 s under load; Cloudflare, measuring from its own network while launching a competitor, reported a 524 ms median. On availability, TechCrunch reported that the API "briefly lost the ability to serve users" during launch week, a 529 "Overloaded" status is part of the documented error set with the recommendation to back off exponentially, and the 1.14 release note says the alias `jev-latest` moves to the new version one week after release, which is why the practice section says to pin versions. The 956M weekly requests are OpenRouter's decisions ranking for the week to October 20, with Jev 1.14 at a further 257M and the next competitor at 38M.

Terms

TypeSafe AI The San Francisco company (founded 2024) that released Jev and coined "System One model".

Artificial Analysis Decision Index An independent leaderboard of decision models reporting accuracy, cost per million tasks and median latency†.

p50 / p95 / p99 Latency percentiles: the median, and the values exceeded by 5% and 1% of requests; the tail is what users notice under load.

529 Overloaded The status code the API returns when the service is temporarily saturated; retry with exponential backoff.

load shedding Rejecting requests under overload rather than queueing them; what the 429 and 529 statuses signal.

OpenRouter A model-routing marketplace that exposes many vendors' models behind one API; it lists decision models under a "decisions" output modality and publishes request rankings.

Intuition

The numbers that made the category are a price with no output tokens, a latency measured in tens of milliseconds, and a billion requests a week within a month of launch. The numbers that should temper the enthusiasm are a 7-second tail, a moving alias, and a vendor evaluation graded by other vendors' models.

6.2 Slide 22: The landscape, October 2026: one category, many insides

OpenRouter lists 22 decision models from 13 makers; the open ones show how different the insides can be.

What the slide shows A table of eight makers and models with what each discloses about its mechanism, its input price per million tokens, its context, its weights and its listing date on OpenRouter: TypeSafe Jev; OpenAI's gpt-6-luna Decisions API; Microsoft-Decision-1; Cloudflare's Clef family; Perplexity's pplx-decider; Liquid AI's d1; Together's Tev1-4B; and Jared Palmer's Kev-4B. A note lists the others and gives the request shares.

Details Within five weeks of Jev's release, OpenRouter's decisions endpoint listed 22 decision models from 13 makers (its guide's count as of October 16; a blog post the same day said 21), reachable through one request shape (`model`, `state`, `questions`) that the vendors adopted as a de facto standard†. The table records what each maker says about how its model works, because that is where the category is most instructive.

TypeSafe Jev 1.13 and 1.14: undisclosed mechanism, "parallel sampler", RLCD; the three production primitives plus Box on 1.14; $0.042; 64k default context; closed weights; listed September 18 and October 14. *OpenAI*: a Decisions API (`/v1/decisions`) in public beta on gpt-6-luna (listed on OpenRouter October 6), with predicate, choice and score question types and a refusal answer type, "about 10× faster than the Responses API", $0.10 per million input tokens with no output charge, a 1.1M-token context, and, notably, no calibration claim at all: the guide tells developers to set thresholds from labelled examples of their own application. *Microsoft-Decision-1*: a post-trained Qwen3.5-9B with single-pass scoring, a 36-benchmark comparison, and a robustness claim of zero flips on option reversal and shuffle (1.3% across eight perturbation types); $0.042; 32k context; no published weights found. *Cloudflare Clef, Clef-flash and Clef-omni*: the most detailed public architecture, a frozen Qwen3.8-27B (or Qwen3.5-9B for flash) backbone with a joint schema head and a rank-256 LoRA, trained with label-smoothed cross-entropy plus a Brier term plus an ordinal partial-credit term described as RLCD-style; omni is a separate fine-tune of the Qwen3-Omni-30B-A3B mixture-of-experts with audio and video input; $0.042 / $0.021 / $0.15 per million input tokens on OpenRouter; 66k context (25k for flash); Apache-2.0 weights on Hugging Face; listed October 1 and 9. *Perplexity pplx-decider v1.2 and flash*: a Qwen3.8-27B backbone with a separate readout head of shape 255×5120 (one row per possible option, read at a marker token after each question), full-attention layers made non-causal so that options can see each other, a fitted temperature of 1.382, and a reported dev-set ECE of 0.014; $0.02 / $0.01; 262k; Apache-2.0; v1.2 listed October 16. *Liquid AI d1 and d1-ultra*: "one forward pass, no tokens generated", "based on a larger open-weight model" with Liquid's customizations, billed per question; $0.02 / $0.042; 66k; listed October 1 and 15. *Together Tev1-4B-experimental*: a supervised fine-tune of Qwen3.5-4B that *generates* one option letter at temperature 0, choice only with 2–24 options, with the full data recipe and training code published; $0.042; 33k; open; listed September 30. *Kev-4B* (Jared Palmer, listed September 25): Qwen3.5-4B-Base, frozen, with a rank-16 LoRA and a pointer head, a block-causal mask that isolates questions from one another, trained on public datasets reshaped into typed requests; $0.042; 8k; Apache-2.0.

The note adds Nace.AI's Drex (v1.5 open under a RAIL-M licence), Inception's Mercury Decide, Upstage's Solar Decide (on its Solar Mini 4 mixture-of-experts), Respan's Span (behaviour probabilities on conversation spans) and ScaleDown. The request shares show how concentrated the market is: for the week to October 20, Jev 1.13 at 956M and 1.14 at 257M against Drex v1.5 at 38M, Decider v1.1 at 37M, Clef at 25M and GPT-6 Luna at 24M, and then Mercury Decide, Drex v2.0 and Clef Flash at 20–22M each, so the two Jev versions are about 85% of the top ten's traffic (1,213M of about 1,429M, derived from the ranking's figures).

Terms

decisions endpoint OpenRouter's `/api/alpha/decisions` (aliased to `/v1/systemone` for TypeSafe's SDKs), the shared request shape across vendors†.

predicate OpenAI's name for a yes/no question returning a probability; the Noul equivalent.

refusal A question type in OpenAI's Decisions API that returns whether a request should be refused.

joint schema head Cloudflare's term for a small transformer over the backbone's final hidden states that emits one logit per allowed option, softmaxed per question†.

readout head Perplexity's separate 255×5120 matrix that maps the hidden state at a question's marker token to one logit per option†.

non-causal attention Attention in which later tokens can be attended by earlier ones; Perplexity makes some layers non-causal so the options of a question can see each other.

pointer head Kev's head, which scores each option's closing token against a special decision token†.

block-causal mask An attention mask that lets each question attend to the state but not to other questions, preserving independence between questions in one pass.

RAIL-M licence A responsible-AI licence with use restrictions; open weights, not open source in the OSI sense.

Intuition

Thirteen makers converged on one request shape in a month, which is how categories form. Under that shape the designs range from "generate a letter" to "a readout head on a frozen 27B model", which is the lecture's point: the interface is the standard, and the insides are still an open design space.

6.3 Slide 23: Open weights: four ways to build one, all on Hugging Face

Read the letters, generate a letter, bolt a head onto a frozen LLM, or fine-tune an encoder; llama.cpp serves them all.

What the slide shows Four panels: (a) a logprob readout from a stock LLM (OpenJev, LLM-as-Jev, AnyJev); (b) a decision head on a frozen LLM (Kev-4B, Perplexity's decider, Clef); (c) an encoder classifier (Laya, Julia-1); (d) generating a letter (Tev1), plus serving in llama.cpp, SGLang and vLLM.

Details The open models that appeared within weeks of Jev fall into four architectures, and seeing them side by side is the best available evidence about what a decision model is†.

(a) Logprob readout. Take a stock generative model, letter the options, ask for one token, and turn the letter logits into probabilities. OpenJev (27B, Qwen3.5-family weights under a non-commercial licence) does exactly this with a fixed calibration, handles at most 52 options per pass, and reports that shuffling the options flips 2.3% of its answers. "LLM-as-Jev" (Li and Wagle 2026, arXiv 2610.02076) formalizes the recipe: prefill "Best answer: [" and sum the log-probabilities of each option's suffix tokens, then softmax; a 4B model reaches 81.4% on JevBench at an ECE of 0.057 with no training, and 84.0% at 0.067 with a LoRA. AnyJev rotates the options across passes and takes the log-mean, with an optional label-prior correction fitted on a few hundred labels, taking a raw Qwen3-8B from 23% order flips and an ECE of 0.240 to 7.7% and 0.095 on a 20-class Banking77 subset (7.3% and 0.184 label-free); Li and Wagle omit permutation averaging to save compute. This is recipe 1 on slide 30.

(b) A decision head on a frozen LLM. Keep the backbone frozen, add a small adapter and a head that produces one logit per option, and train on typed requests with a proper scoring rule. Kev-4B is the fully documented example: Qwen3.5-4B-Base, a rank-16 LoRA, and a pointer head that scores each option's closing token against a `<decide>` token, 33.8M trainable parameters in all, trained on public datasets converted into TypeSafe-shaped requests with no Jev outputs used; its model card's comparison with Jev is honest: MMLU-Pro 0.565 against 0.840 and a Brier score of 0.243 against 0.211 on its transfer development

set (an older README reports 0.328 and a confident-error rate of 8.2% against 3.7% for an earlier version). Perplexity's decider and Cloudflare's Clef are the same idea at 27B with the published training losses of slide 22. This is recipe 2.

(c) An encoder classifier. Laya is a 421M ModernBERT-large encoder with a two-layer head that scores each option at its own `[MASK]` token; its card claims RLCD-style training with proper scoring rules and REINFORCE; it is Apache-2.0 and was downloaded 54k times in a month. Its numbers are contested: the card's own table shows it ahead of Jev on one typed-decisions set and far behind on Banking77 (0.425 against 0.870, on slightly different label subsets), independent runs place it 20th of 23 on the S1MB benchmark and below the majority baseline on Archestra's agent-call test, and a Hacker News thread accused the card of target leakage. Julia-1 is a 144M mmBERT-small sibling. This is the family Raschka means by "the plug-and-play version of dedicated classifiers", and it is recipe 3.

(d) Generate a letter. Together's Tev1-4B-experimental is a supervised fine-tune of a 4B model that emits one option letter: a generator in a decision model's clothes, with no head and no calibration objective, included because its recipe and training code are public.

Serving caught up as fast as the models. llama.cpp merged support for a `/v1/systemone` endpoint on October 2 (PR 29818) with GGUF conversions under ggml-org, and reports per-question medians on an RTX PRO 6000 of 3 ms for Julia-1, 5 ms for Laya, 12 ms for Kev-4B, 36 ms for a 4B head model and 43 ms for OpenJev 27B; SGLang and vLLM serve Clef on the same Jev-compatible endpoint. Two community leaderboards exist and disagree: the Jev Decision Index, a Hugging Face Space of open reproductions of the vendor's benchmark, places Perplexity's decider v1.2 (68.1) above Jev (60.1, ECE 0.074) and the 27B member of the Kev family (58.8); the S1MB benchmark (137 tasks) places Jev 1.13 first, with its author's own model second. Both are young, and the lecture treats them as indicative.

Terms

OpenJev A 27B open model that reads option probabilities from letter logits with a fixed calibration; non-commercial licence†.

LLM-as-Jev Li and Wagle (2026): training-free typed decisions by summing option-suffix log-probabilities after a prefilled answer prompt.

AnyJev A training-free readout that rotates options across passes and corrects the label prior to reduce order flips and ECE†.

Kev Jared Palmer's open reimplementation: a frozen 4B base, a rank-16 LoRA and a pointer head, trained on public data reshaped into typed requests†.

Laya A 421M ModernBERT-large decision model with a two-layer head per option; Apache-2.0; contested benchmarks†.

ModernBERT Warner et al. (2024): a modern bidirectional encoder trained on 2T tokens with an 8,192-token context; the encoder backbone of Laya and of recipe 3.

GGUF llama.cpp's single-file weight format; the open decision models were converted to it for the `/v1/systemone` endpoint.

Jev Decision Index / S1MB Two community leaderboards for decision models on Hugging Face; they disagree about the ordering of the top models†.

Intuition

Same door, four rooms: peek at a chatbot's letter logits, bolt a head on a frozen model, fine-tune an encoder, or just ask for a letter. The first is an afternoon's work and gets most of the way; the second is what the serious open models do; the third is what you did before any of this existed.

7 Limits (slides 24–25)

7.1 Slide 24: Jaggedness: where a decision model is rough

The vendor publishes its own list; independent tests confirm it and add wording sensitivity.

What the slide shows Two panels: the vendor's own jaggedness list for Jev 1.13 (literal reading; counting, arithmetic and date comparison; double negatives; irrelevant context; adversarial content treated as data; contradictory criteria; a lean toward the first option; no text), and what others measured (order, wording, literalness). A closing paragraph on the literature that predicted most of it and the mitigations that transfer.

Details The vendor's jaggedness page[†] is unusual in that it exists: a list of known rough edges, dated and versioned, many "expected to be fixed later". Its items. *Literal reading*: criteria are taken at their word, so a criterion that assumes common sense will be applied without it. *Counting, arithmetic, date comparison*: unreliable; compute these in code and pass the result in the state. *Double negatives and indirection*: hurt accuracy. *Irrelevant context* ("context rot"): material in the state that the questions do not need measurably lowers accuracy, which is the basis of the "send only what the questions need" rule of slide 9. *Adversarial content treated as data*: instructions embedded in the state can steer the answer, the subject of slide 25. *Contradictory criteria*: give arbitrary answers. *Option order*: Jev 1.13 "leans toward the option that comes first". *No text*: the model cannot say why. (A related structural point raised by independent testers rather than the vendor: a Noul and its negation, asked as two questions, need not sum to 1, because each is evaluated independently.)

Independent measurements confirm the list and add wording sensitivity[†]. *Order.* Anaconda reversed the options on 80 cases: 79 kept the same answer, but probabilities moved by up to 0.20, and identical requests repeated varied by up to 0.04. Archestra found about 4 of 100 agent-approval decisions changed on option reversal, a drop of one to two points. Open readouts are far worse: in Archestra's hands a 0.6B model always chose the first option and another always the last; AnyJev measured 23% flips on a raw Qwen3-8B. *Wording.* TypedBench (Sharma, Duncan, Marceau Caron and Vollmer 2026, arXiv 2610.11392), on 500 support tickets, found a median accuracy of 0.874 but a range of 0.708 to 0.946 across paraphrases of the same question, and characterized Jev as "wording-sensitive and systematically underconfident". *Literalness.* A 7,805-request behaviour study found that changing "5-minute walk" to "5-minute drive" in a question flipped the answers on 0 of 20 to 20 of 20 cases, and that letter counting was unreliable, confirming the vendor's own items.

The literature predicted most of this before the product existed. Zheng et al. (2023) showed that multiple-choice selection bias is mostly a token prior on the option identifiers and gave a label-free fix (PriDe); Pezeshkpour and Hruschka (2023) measured 13–75% performance swings from reordering options; Wang et al. (2023) showed judges flipping verdicts when candidate positions are swapped. The mitigations that transfer to decision models are the same: shuffle the options, or rotate them across passes and average (AnyJev's recipe); correct the label prior; list "none of these" so that the forced distribution has an honest home; and keep the state short.

Terms

jaggedness The vendor's term for the uneven profile of a model that is strong on some kinds of question and unreliable on others; also used for frontier models generally.

literal reading Applying criteria exactly as written, without the common-sense gloss a person would add.

double negative A question or criterion phrased with two negations; degrades accuracy.

structural invariant A relation that should hold between answers (a Noul and its negation summing to 1) but is not enforced because questions are answered independently.

order sensitivity Change of answer or probability when the options are reordered; measured by the flip rate.

flip rate The fraction of cases whose argmax changes under a perturbation (reversal, shuffle, paraphrase, added context).

wording sensitivity Variation of accuracy across paraphrases of the same question; TypedBench's 0.708–0.946 range.

TypedBench A benchmark (Sharma et al. 2026, arXiv 2610.11392) of typed decisions on support tickets that measures wording sensitivity and cost-weighted decision quality.

Intuition

A decision model is a very fast literalist. It will do exactly what the criteria say, count badly, be swayed by the order of the menu, and be distracted by anything you put in front of it that it did not need. Every one of those is a known trait of its generative ancestors; the vendor's list is a courtesy, and the tests are the confirmation.

7.2 Slide 25: What a calibrated probability cannot tell you

Calibration is about a population of questions; the state may be written by an adversary; the label is not the outcome.

What the slide shows Four panels: calibration is relative to a distribution (the fair-coin test); the state is attacker-controlled (JevOut, JevAdvBench); the label is not the outcome (Anaconda's CFPB study); and confidence is a shape, not a truth (Guo et al. 2026, OpenRouter, a zero-mass case).

Details *Relative to a distribution.* The cleanest critique of the "calibrated" claim came from a blog post titled "Jev can't be calibrated"[†]: ask the model which side a fair coin landed on and it answers 0.92 for one side; on 400 die rolls (a Hacker News reproduction) it put about 83% on one face. There is nothing in such a request from which any model could learn the base rate, so no answer can be calibrated for it; calibration is a property of a model over a distribution of questions, and the vendor's claim can only be about its own distribution. Yours is different, and slide 27 shows the consequence: independent ECE measurements on Jev range from about 0.02 (the pooled raw ECE in the crash-narrative study, before recalibration) to 0.25 (Jev on the tasks of a competing model card), with 0.06, 0.15 and 0.17 in between on other tasks.

Attacker-controlled state. Because the model treats adversarial content as data (the vendor's own item), anything written by someone else in the state is an attack surface. JevOut (Xu et al. 2026, arXiv 2609.30243) found that fluent, natural-looking additions to the context flipped between 61.4% and 73.2% of initially correct decisions across four decision systems and seven datasets, pushing the wrong option above 0.7. JevAdvBench (Hu et al. 2026, arXiv 2609.31142) ran 9,744 single-edit black-box attacks and found that an appended unverified opinion flips 12.1% of decisions and drags 38% of confident answers below a 0.8 review threshold. Support tickets, emails and web pages are state written by someone else, and the defences belong in the harness: deny-lists and schema checks in front of the model (slide 28), and a review band that

an attacker must push through rather than a single threshold.

The label is not the outcome. Anaconda† scored Jev on 2,400 CFPB consumer complaints against what actually happened to each complaint. On the resolution outcome, the one label that is a real outcome, the zero-shot model scored 45%, below the majority class, and 55% after a base-rate correction, while an XGBoost on word counts led. On a rule-defined quick-review question, writing the bank's own rules into the question raised the model to 82%, but the rules alone, applied in code, scored 94%. The model answers the question you wrote, and its calibration is to that question's labels; if the outcome you care about is determined by rules and histories the state does not contain, no calibration will help, and a classical model with the right features will win.

Confidence is a shape. The confidence field is peakedness (slides 6–7), computed from the distribution, and a peaked distribution can be peaked on the wrong option. Guo et al. (2026) found a good ranker (AUROC 0.886) whose per-benchmark ECE (median 0.168) is far from the 0.074 expected under perfect calibration even though its pooled ECE is 0.047; OpenRouter's Banking77 analysis said Jev "overestimated its own accuracy mid-range"; one pilot found zero probability mass on the true label in 16% of emotion-classification cases. And because there is no text, there is no trace: an audit of a decision needs a second system that can explain, which is one more argument for the cascade.

Terms

base-rate blindness The impossibility of calibrating an answer whose base rate is absent from the request; the fair-coin test.

prompt injection Instructions placed in content the model will read, intended to steer its output; for a decision model, instructions in the state.

JevOut Xu et al. (2026): natural context additions that flip a majority of initially correct decisions.

JevAdvBench Hu et al. (2026): 9,744 single-edit black-box attacks on decision models.

review band A range of probabilities routed to a person rather than acted on; a defence an attacker must push an answer through.

outcome label A label defined by what actually happened rather than by a judgment of the text; Anaconda's CFPB resolution outcomes.

zero-mass failure A case in which the true label receives probability near zero; the worst kind of confident error, and unbounded under log loss.

Intuition

A calibrated probability tells you how often answers like this one are right, over questions like the ones the model was trained on, when nobody is trying to fool it, about the label you asked for. Each clause is a place it can fail, and the four panels are the four clauses.

8 Evaluation (slides 26–27)

8.1 Slide 26: Evaluating a decision model: six questions, six numbers

Accuracy alone is the wrong test for a model whose product is a probability; test the probability and the selector.

What the slide shows A table of six questions, each with the measure that answers it and the pitfall to avoid: is it right (accuracy, macro-F1, agreement); is the probability honest (ECE, Brier, log loss, reliability

diagrams); does it know when it is wrong (AUROC, AURC); is it stable (flips under shuffle, paraphrase and added context; repeat variance); what does it cost (dollars per thousand, latency percentiles); does it move the business number (cost-weighted error, net benefit). A method line.

Details The table is the evaluation protocol the lecture recommends, and each row has a reason. *Is it right?* Accuracy or macro-F1 against human labels, with Cohen's κ when the labels are judgments; always beside the majority baseline, which was 79% in Archestra's agent-call test and makes a 93% look different; and with the understanding that label noise caps every number in the row. *Is the probability honest?* ECE with adaptive (equal-mass) bins, the Brier score, log loss, and a reliability diagram per question type; the pitfalls are that ECE depends on the binning (Nixon et al. 2019 show the choice of metric re-ranks methods and recommend adaptive bins and class conditioning), that ECE is discontinuous as a function of the predictions (Błasiok et al. 2022 propose consistent alternatives), and that for Choice questions with many options a class-wise ECE (Kull et al. 2019) catches miscalibration on minority options that the top-label ECE hides. *Does it know when it is wrong?* The AUROC of confidence against correctness and the risk–coverage curve with its AURC (slide 17); the pitfall is to conflate this with calibration, since a well-ranked miscalibrated model is still useful behind a tuned threshold. *Is it stable?* Flip rates under option shuffle, paraphrase and added context, and the variance over repeated identical calls; the measured values on slides 24–25 (0.708–0.946 across paraphrases, 0.04 across repeats, 12–61% under injected text) are the scale to expect. *What does it cost?* Dollars per thousand decisions and the p50 and p95 latency at your own concurrency, remembering the 7-second tail and that an alias moving to a new version changes every other row. *Does it move the business number?* The cost-weighted error at your thresholds (Elkan 2001) and a net-benefit curve (Vickers and Elkin 2006); TypedBench's cautionary result is that with underconfident probabilities the cost-weighted Bayes action cost 2,300 per 10,000 cases against 1,080 for the plain argmax, so recalibration (slide 16) must come before cost-sensitive decisions.

The method is short: a few hundred labelled cases per question type from your own traffic, held out from any threshold tuning; the whole table rerun on every model version and every alias move; and the raw probability vectors kept, so that any row can be recomputed later with a different metric or a different threshold.

Terms

macro-F1 The F1 score averaged over classes with equal weight; robust to class imbalance.

Cohen's κ Agreement between two labellers corrected for chance; the right measure when the labels are judgments.

majority baseline The accuracy of always predicting the most common class; the floor every reported accuracy must be compared with.

adaptive binning Equal-mass rather than equal-width bins for ECE; reduces the dependence of ECE on bin boundaries (Nixon et al. 2019).

cost-weighted error Expected cost at the chosen thresholds, with each kind of error weighted by its cost; the number the business sees.

Bayes action The action that minimizes expected cost under the model's probabilities; only as good as the calibration of those probabilities.

held-out set Labelled cases not used for tuning thresholds or calibration, so that the evaluation is not optimistic.

Intuition

Grade a decision model the way you would grade a weather forecaster: not only how often the forecast was right, but whether 70% meant 70%, whether the forecaster knew which days were hard, whether the forecast changed when you asked the same question twice, and what the forecasts cost you when you acted on them.

8.2 Slide 27: What the independent evidence says, five weeks in

Strong and cheap at discrimination, a few points under the best generators, calibration that depends on the task.

What the slide shows A nine-row table of independent results on Jev: Banking77 intents, IMDb sentiment, prompt injection, agent tool-call approval, faithfulness and summary judging, alignment-failure detection, police crash narratives, support tickets (TypedBench), and CFPB complaints against real outcomes; for each the Jev result, the comparison, and who ran it. A closing pattern.

Details The rows, with their sources[†] where not papers. *Banking77* (OpenRouter): 3,080 utterances over 77 intents; Jev 1.13 at 81.0% (confidence interval 79.6–82.3) against Claude Opus 5 at 84.4%; $0.11 against $2.42 per thousand; answers with confidence at least 0.99 cover 58% of cases at 96.3% accuracy while those below 0.5 are 3.5% of cases at 29.6%; a cascade that sends cases below 0.90 to Opus reaches 84.0% for $0.69 per thousand. *IMDb* (Raschka): 25,000 reviews, 96.47% with a Choice question, $0.65 in total, 22 minutes; a fine-tuned ModernBERT reached "similar accuracy". *Prompt injection* (Dibia): 662 messages; the Jev figures, 96.5% at ECE 0.059 with a task description, are a third-party run Dibia cites and calls "indicative only" because the prompts differ; Dibia's own stock Qwen2.5-7B scored by option logprobs reached 93.0% at ECE 0.061, and one fitted temperature took the 7B model's ECE from 0.10 to 0.03. *Agent tool-call approval* (Archestra): 100 real coding-agent calls; Jev 93% zero-shot and 95% with nine examples against Claude Sonnet 5 at 98% and a majority baseline of 79%; reversing the options changed about 4 of 100 decisions. *Faithfulness and summary judging* (OpenRouter): 138 items in two parts; on the 88 HaluEval faithfulness items Jev scored 84 of 88 (95.5%) against GPT-5.6 Luna's 83 of 88 (94.3%), Brier 0.043 against 0.054, median latency 171 ms against 1,662 ms; on the 50 SummEval summaries, scored on an open rubric, Jev's Spearman correlation with humans on consistency was 0.47 against the generator's 0.72 (0.42 against 0.44 on coherence). *Alignment-failure detection* (Guo et al. 2026, arXiv 2609.29429): 44 benchmarks, 7,193 instances; median AUROC 0.886 zero-shot over the 31 yes/no benchmarks, at 63× lower cost than LLM judges; a median per-benchmark ECE of 0.168 against 0.074 expected under perfect calibration, with a pooled ECE of 0.047. *Police crash narratives* (Rafe and Das 2026, arXiv 2609.24052): 195,857 narratives and 27 questions at $0.154 per thousand; F1 0.908 against 2,416 human labels; raw probabilities overstated prevalence and recalibration cut the error 3.3×. *TypedBench* (arXiv 2610.11392): 500 tickets; median 0.874; 0.708–0.946 across wordings; systematically underconfident. *CFPB complaints* (Anaconda): 2,400 complaints; on the real resolution outcome 45% zero-shot, below the majority, and 55% at best, behind an XGBoost on word counts; on a rule-defined quick-review question 82% with the bank's rules written in, against 94% for the rules alone.

The pattern the deck draws. On discrimination (accuracy, F1, AUROC) the model lands close to frontier generators at a few percent of their cost and a tenth of their latency, and close to a fine-tuned small model on narrow tasks. On calibration the picture is mixed: an ECE of 0.02–0.06 on some tasks and 0.17–0.25 on others, overconfident in one study and underconfident in another, so recalibrate on your own labels. On robustness, injected text moves it (slide 25). The table on slide 26 is the contract, and the vendor's calibration claim is the starting point, not the conclusion.

Terms

Banking77 A 77-intent banking-query classification benchmark (3,080 test utterances) used by OpenRouter's comparison.

HaluEval / SummEval Benchmarks of hallucination (faithfulness) judgments and of summary quality ratings used in OpenRouter's judge comparison.

Spearman correlation Rank correlation between a model's scores and human ratings; the standard measure for judge agreement on rubrics.

null baseline (calibration) The calibration error of a trivial predictor, such as one that always returns the base rate; a model can be worse calibrated than it.

CFPB complaints Public consumer complaints to the US Consumer Financial Protection Bureau with recorded resolutions; Anaconda's outcome-labelled test.

few-shot (decision model) Including labelled examples in the question's criteria; Archestra's nine examples raised accuracy from 93% to 95%.

Intuition

Five weeks of independent testing say: it classifies about as well as the best generators for a hundredth of the price, it ranks its own errors well, and its probabilities mean what they say on some tasks and not on others. That is a very good classifier with an unfinished calibration claim, and it should be used as one.

9 Practice (slides 28–30)

9.1 Slide 28: One decision, as it goes over the wire

A coding agent asks whether to run a shell command: two Nouls, a Score, a deny-list in front, and thresholds in code.

What the slide shows A full request: model `jev-1.13.0`; a state with the task ("make the unit tests pass"), the working directory, the tool and the command (`rm -rf build/ && npm test`); three questions: `reversible` (Noul: can the command be undone without data loss?), `serves_task` (Noul: does it plausibly advance the task?), and `blast_radius` (Score over four levels: none, local files, repo-wide, outside the repo). The response: 0.93, 0.95, and a score of 1.1 with confidence 0.8; about 290 input tokens, roughly a hundredth of a cent. Two panels: the code around the call, and where the pattern already runs.

Details The example follows the OpenRouter cookbook for auto-approving a coding agent's permission prompts[†], which is the most concrete published use of a decision model inside an agent. Its structure is the lecture's whole argument in miniature. First, a static deny-list is checked before the model is called, and the cookbook is explicit that "the static risk list, not the threshold, is the security boundary": anything a regular expression can catch (a recursive delete of the home directory, a credential file, a network call to an unknown host) should be caught without a probability. Second, the model is asked narrow questions whose answers are probabilities: in the cookbook, two Nouls, `reversible` and `serves_task`, approved only when both are at least 0.9; the slide adds a blast-radius Score because the deck has been arguing for cost-sensitive thresholds and a four-level severity is the natural input to one. Third, the thresholds live in code, in the form of slide 18: run when both Nouls are at least 0.9 and the blast radius at most 1.5 (local files at worst); ask a person when reversibility is below 0.5 or the blast radius at least 2.5 (outside the repo); otherwise ask the user to confirm. Fourth, the call fails closed: on a timeout (8 s in the cookbook, which then falls back to

the agent's normal permission prompt; Cursor's hook has an explicit fail-closed flag) the decision goes to a person, never to "run". Fifth, the three distributions are logged with the eventual outcome, because that log is the calibration set of slide 16 and the evaluation set of slide 26.

The response numbers are illustrative of the cookbook's own sample (its `bun test` example scored 0.93 and 0.95 at a cost of $0.0000168); the cost on the slide is derived from 290 input tokens at $0.042 per million. The score of 1.1 for `rm -rf build/ && npm test` says the model puts most of its mass on "local files", which is right: the command deletes a build directory and runs tests.

Where it runs[†]. OpenRouter's cookbook ships hooks for Claude Code and Codex (`PermissionRequest`), OpenCode (`permission.hook`) and Cursor (`beforeShellExecution`). Vercel's agent framework exposes `approval: auto()`, which classifies a tool call as clear or caution with a decision model. LangChain's integration provides a classifier, a model-router middleware that picks a fast or a powerful model per request, and an `AutoModeMiddleware` guardrail for tools such as `bash`; the OpenClaw harness exposes `decisions.evaluate` for tool pre-filtering and respond-or-not gating. All of these are weeks old and described by their authors as early; all of them have the same shape: code decides, the model scores.

Terms

permission prompt A coding agent's request for the user's approval before running a tool; the decision being automated.

deny-list A static list of patterns that are blocked before any model is consulted; the security boundary of the cookbook.

fail closed Treating a timeout or an error as a refusal rather than an approval.

blast radius The worst-case scope of a wrong action; the slide's Score levels: none, local files, repo-wide, outside the repo.

hook An extension point in an agent harness (`PermissionRequest`, `beforeShellExecution`) at which a decision can be inserted.

decision log The record of request id, model version, full probability vectors, threshold fired and outcome; the raw material for recalibration and evaluation.

Intuition

The model never decides anything. It grades three questions in a hundred milliseconds; a regular expression decided before it, three numbers in code decide after it, and a timeout decides when it is silent. That is what "code in control" means in practice.

9.2 Slide 29: Writing questions that work

What the docs advise, what the independent tests add, and what to log so you can prove it later.

What the slide shows A Do panel (decompose; write the rules in; describe options; batch; compute in code) and a Don't panel (don't send the whole history; don't trust option order; don't threshold on the label; don't ship `jev-latest`; don't let the state carry instructions), and a closing line on what to log.

Details The Do side. *Decompose.* One judgment per question; a broad question hides several. The most striking published example[†] is a community phishing benchmark on 2,000 synthetic emails: a single "is this phishing?" question scored 62.6% (behind a small generative model's 81.3%), while five sub-questions (sender legitimacy, urgency, link mismatch, and so on) combined by a logistic regression fitted on 1,000 labelled emails scored 95.0%, ahead of the generator's five-signal 93.2% (a gap the author found not

significant) at a twelfth of its cost; a two-line regular expression scored 91.8% on the same data, which is the other lesson. *Write the rules in.* Anaconda's rule-defined quick-review question (slide 25) reached 82% once the bank's own rules were part of the question, and the rules alone scored 94%: what is deterministic belongs in code, and what the model cannot know, the state or the question must say. *Describe options* with `what`, `not_for` and `examples`, levels with signals, and both sides of a Noul (slides 6–8). *Batch* every question about a state into one call, speculative ones included (slides 9 and 20), and do the arithmetic (dates, counts, amounts) in code first, passing the results in as fields.

The Don't side. *Don't send the whole history* when the question is about the last message; irrelevant state lowers accuracy (slide 24). *Don't trust option order*: shuffle, or rotate and average, and at minimum measure your own flip rate. *Don't threshold on the label*: keep the full probability vector, set thresholds on it from costs (slide 18), and recalibrate it on your labels (slide 16). *Don't ship an alias*: pin `jev-1.13.0` rather than `jev-latest`, and rerun the slide-26 table when you move; the vendor's alias moves one week after each release. *Don't let the state carry instructions*: it is untrusted input, and deny-lists and schema checks go in front of the model (slide 25).

The closing line is the one most often skipped. Log the request id, the model version, the full `probabilities` for every question, the threshold that fired and the eventual outcome. Without that log there is no reliability diagram, no recalibration, no risk–coverage curve, and nothing to show an auditor who asks why the system acted.

Terms

decomposition Replacing one broad question with several atomic ones whose answers are combined in code.

sub-question ensemble Several Nouls combined by a learned or hand-set rule (a logistic regression in the phishing example) into one decision.

version pinning Naming a specific model version in production rather than a moving alias.

untrusted input Any state content written by someone other than the application; the default assumption for tickets, emails and web pages.

schema check A deterministic validation of a tool call or an extraction against an allowed structure, run before or instead of a model.

Intuition

Ask small questions, put the facts in front of the model and the rules in the question, keep the arithmetic and the thresholds in code, pin the version, and write everything down. None of it is specific to one vendor; all of it is what the evidence says separates 60% from 95%.

9.3 Slide 30: Build your own: three recipes, in increasing effort

A readout with a temperature, a head on a frozen model, or a fine-tuned encoder; the vendor's number is the floor.

What the slide shows Three cards: (1) a logprob readout with a fitted temperature, an afternoon's work; (2) a decision head on a frozen LLM, about a week; (3) a fine-tuned encoder, when you have labels. A closing line on evaluating whichever you build against the vendor's model.

Details *Recipe 1: readout plus temperature.* Serve any open generative model in vLLM or llama.cpp. Letter the options, prefill "Best answer: [" and read the option log-probabilities in one prefill with no decode (Li and Wagle 2026: a 4B model reaches 81.4% on JevBench at ECE 0.057 with no training at all). Rotate the

options across a few passes and average to remove position bias (AnyJev). Fit one temperature on 300–1,000 labelled cases from your own traffic (Dibia's 7B went from ECE 0.10 to 0.03†). The cost is one prefill per state and the latency tens of milliseconds on a GPU; there are no output tokens here either.

Recipe 2: a head on a frozen LLM. Freeze a 4–27B base model, add a LoRA and a pointer or schema head that scores each option at a marker token, and train with cross-entropy plus a Brier term, with ordinal partial credit for Score levels, on public datasets reshaped into typed requests; Kev-4B does this with 33.8M trainable parameters and Clef with a rank-256 LoRA and the published loss (slide 22). Serve on llama.cpp's `/v1/systemone` endpoint at 12–43 ms per question on a workstation GPU†. What you get over recipe 1 is batching of many questions in one pass, independence between questions by construction, and a model you own; what it costs is a week and a GPU.

Recipe 3: a fine-tuned encoder. ModernBERT or DeBERTa with a classification head, trained by cross-entropy on your labels and temperature-scaled afterwards. Raschka's best ModernBERT came within about a point of Jev on IMDb ("similar accuracy"); Red Hat's AI safety team found a 200M DeBERTa prompt-injection classifier ahead of Jev, 89.0% to 86.4%, at a median of 54 ms on a laptop CPU against 348 ms for the API†. With few labels, SetFit (Tunstall et al. 2022); with none, distil from a generative judge: Pangakis and Wolken (2024) found classifiers trained on GPT-4 labels as good as those trained on human labels across 14 tasks, and Bucher and Martini (2024) that fine-tuned small models still beat zero-shot generators on classification. This is the fastest and cheapest at inference and has the oldest tooling; its limit is a fixed label set, so a new question means retraining.

Whichever you build, run the slide-26 table against the vendor's model on the same held-out set. The evidence so far (slide 27) is that a tuned 7B readout or a fine-tuned 200M encoder lands within a few points on narrow tasks, and that the hosted model wins on breadth and on zero-shot new questions. Breadth is what the $0.042 per million tokens buys; whether you need it is a question about how many different questions you will ask.

Terms

prefill-only inference A forward pass over the prompt with no generated tokens; what a logprob readout and a decision head both cost.

option rotation Asking the same question several times with the options cyclically shifted and averaging the probabilities by option; removes position bias.

LoRA Low-rank adaptation: small trainable matrices added to a frozen model's weights; rank 16 in Kev, rank 256 in Clef.

marker token A special token after a question (or an option) at which a head reads the hidden state to produce that question's (or option's) logit.

ordinal partial credit A loss term that rewards probability on levels adjacent to the true level less than on the true level but more than on distant ones; Clef's training uses one.

SetFit Tunstall et al. (2022): contrastive fine-tuning of a sentence encoder plus a head, effective from a handful of labels per class.

distillation from a judge Training a small classifier on labels produced by a large generative model; Pangakis and Wolken (2024).

Intuition

An afternoon gets you a readout that is a few points behind; a week gets you a head that is yours; and if you already have labels, the thing you would have built in 2022 still works. The hosted model's advantage

> is that it answers questions you have not thought of yet.

10 Wrap-up (slides 31–32)

10.1 Slide 31: Summary: what to take from this lecture

Calibrated classification as a service: the science is old, the economics are new, the evidence is young.

What the slide shows Eight things to remember and four open questions as of October 2026.

Details The eight points, with the slides they compress. (1) The output is a distribution over options you supplied; the probability, not the label, is the product (slides 2, 5–9). (2) Confidence summarizes that distribution's shape and is not a correctness guarantee (slides 6–7, 25). (3) Calibration is a population property, scored by proper scoring rules; preference tuning destroys it, which is why a decision model is a separate model with a separate objective (slides 10–15). (4) Thresholds come from the cost of each mistake and live in code (slide 18). (5) Selective prediction and conformal sets turn a probability into act, confirm or escalate (slides 17, 19). (6) Cascades put the cheap calibrated model in front of the expensive one (slide 20). (7) Recalibrate and red-team on your own data; the state is untrusted (slides 16, 24–27). (8) The same thing can be built open, as a readout, a head or an encoder (slides 23, 30).

The four open questions. *Is RLCD more than cross-entropy plus Brier?* There is no paper; the open reimplementations that use exactly that loss get close on narrow tasks and fall short on breadth, which is consistent with either answer. *Can calibration survive injected context?* Flip rates of 12–73% under adversarial additions say not yet; the honest fix may be in the harness (deny-lists, review bands, schema checks) rather than in the model. *What is the right benchmark?* Accuracy leaderboards reward guessing; the two community indices disagree with each other and with the vendor; a benchmark that reports calibration, ranking, stability and cost together (slide 26) does not yet exist as a leaderboard. *Who owns the decision?* Every vendor says "code decides"; the thresholds, deny-lists and logs are where that promise is kept or broken, and they are the application's, not the model's.

Intuition

A decision model is a cheap, fast, calibrated-on-its-own-terms classifier with a general question interface. Use it for what it is, measure what it claims, and keep the decision in code.

10.2 Slide 32: Sources

Documentation and model cards, papers, and the independent tests quoted; † marks vendor and community material.

What the slide shows Two columns: vendor and platform documentation, independent tests, and the papers by theme. The full list with arXiv identifiers is Appendix D and the paper kit.

Details The deck's sources fall into three classes, and the reader should weigh them differently. *Primary documentation and model cards* (TypeSafe's docs, OpenRouter's guides and cookbooks, the OpenAI, Microsoft, Cloudflare, Perplexity, Liquid and community model cards, the llama.cpp announcement) are authoritative about what a product does and claims, and silent or self-interested about how well it works. *Independent tests* (Anaconda, Raschka, Dibia, Archestra, the Artificial Analysis index, the community benchmarks, and the arXiv papers of Rafe and Das, Guo et al., Hu et al., Xu et al., Wu and Lim, Li and Wagle, and Sharma et al.) are the evidence about how well it works; most are small, several are weeks old, and some have conflicts of interest the deck notes (OpenRouter resells Jev; Cloudflare measured Jev while launching

Clef; a benchmark author's own model placed second on his benchmark). *Papers* on calibration, scoring rules, RL for calibration, selective prediction, conformal prediction, LLM classifiers and judges, cascades, dual-process AI and distillation are the stable part, and the reading list in Appendix D gives one line on why each is worth reading. Where the deck quotes a number from a secondary source it carries a dagger; where it computed one it says derived; and where a vendor's claim is the only source, the notes say so in the text.

Intuition

Read the documentation for what the thing is, the independent tests for what it does, and the papers for why. The first will change by next month, the second is growing weekly, and the third is the part of this lecture that will still be true in five years.

A Notation

symbol	meaning	notes
p, p_i, p_{ik}	a returned probability	for an option, a level, or class k of example i
$p_{\max}$	the largest Choice probability	the probability of the `choice`
n	number of options or levels	1–255 for Choice, 2–10 for Score
m	the most likely Score level	the anchor of the Score confidence
$\mathrm{MAD}_{\mathrm{unif}}$	spread of a uniform distribution over levels	$\frac{1}{n}\sum_i \lvert i - \frac{n-1}{2} \rvert$
y	a label or outcome	one-hot $\mathbb{1}[y = k]$ in the Brier score
K	number of classes in the Brier sum	also the beam width ($K = 3$) in the hierarchy cookbook
N, B, n_b	evaluation set size, number of bins, bin count	in ECE and the Brier and log-loss averages
ℓ	a loss function	$\ell(f(x), y)$ in the selective risk
q	a reported probability	in the scoring-rule argument; $q = p$ is honest
$S(q, y)$	a scoring rule	loss charged to a report once y is known
ECE	expected calibration error	bin-weighted mean $\lvert \mathrm{acc}_b - \mathrm{conf}_b \rvert$
T	temperature	divides the logits before the softmax
z_i	a logit	pre-softmax score for option i
(f, g)	a selective classifier	predictor and selector
ϕ, R	coverage, selective risk	fraction answered, error among the answered
τ	a confidence threshold	$g = \mathbb{1}[\mathrm{conf} \geq \tau]$
c_{FP}, c_{FN}, p^*	costs and the optimal threshold	$p^* = c_{\mathrm{FP}} / (c_{\mathrm{FP}} + c_{\mathrm{FN}})$
α, $\hat{q}$, $C(x)$	conformal miscoverage, quantile, prediction set	split conformal prediction
s_i	a nonconformity score	$1 - \hat{p}(y_i \mid x_i)$
†	secondary source	vendor docs, model cards, blogs, community benchmarks, press

B Formulas at a glance

Choice confidence. $(p_{\max} - \frac{1}{n})/(1 - \frac{1}{n})$; 0 at uniform, 1 at certainty; depends on $p_{\max}$ only. $(0.62, 0.21, 0.12, 0.05)$: $(0.62 - 0.25)/0.75 = 0.49$.

Score. expected level $\sum_i i\, p_i$; confidence $\max\big(0,\, 1 - \sum_i p_i |i - m| / \mathrm{MAD}_{\mathrm{unif}}\big)$ with m the mode and $\mathrm{MAD}_{\mathrm{unif}} = \frac{1}{n}\sum_i |i - \frac{n-1}{2}|$ ($= \frac{2}{3}$ for $n = 3$, $= 1$ for $n = 4$, $= 1.2$ for $n = 5$). $(0, 0.57, 0.43)$: score 1.43, confidence $1 - 0.43/0.667 = 0.35$.

Noul. a single $p = \Pr(\text{yes})$; certainty $|2p - 1|$, equal to the two-option Choice confidence.

Request cost (derived). input tokens $\times$ \$0.042 per million; output free. 10k tokens, 20 questions: about \$0.0004.

ECE. $\sum_b \frac{n_b}{N} |\mathrm{acc}_b - \mathrm{conf}_b|$ over B bins; use equal-mass bins and report B.

Brier. $\frac{1}{N}\sum_i \sum_k (p_{ik} - \mathbb{1}[y_i = k])^2$; Murphy: reliability $-$ resolution $+$ uncertainty.

Log loss. $-\frac{1}{N}\sum_i \log p_i(y_i)$; cross-entropy; strictly proper; unbounded.

Proper scoring. expected Brier $p(1-q)^2 + (1-p)q^2$, minimized at $q = p$; expected log loss $-p \ln q - (1-p)\ln(1-q)$, same; 0/1 loss of the argmax is flat on each side of $q = 0.5$.

Temperature scaling. $p_i = \exp(z_i/T)/\sum_j \exp(z_j/T)$, T by minimizing held-out log loss; argmax unchanged. $z = (2, 1, 0.2)$: $T = 1$ gives $(0.65, 0.24, 0.11)$, $T = 2$ gives $(0.50, 0.30, 0.20)$, $T = 0.5$ gives $(0.86, 0.12, 0.02)$.

Selective prediction. coverage $\phi = \mathbb{E}[g]$, risk $R = \mathbb{E}[\ell\, g]/\phi$, $g = \mathbb{1}[\mathrm{conf} \geq \tau]$; AURC $=$ area under $R(\phi)$.

Cost threshold (Elkan). act iff $(1-p)c_{\mathrm{FP}} \leq p\, c_{\mathrm{FN}}$ iff $p \geq p^* = c_{\mathrm{FP}}/(c_{\mathrm{FP}} + c_{\mathrm{FN}})$. Costs $1:50$ give 0.02; $49:5$ give 0.91; $2:2$ give 0.50; $1:20$ give 0.05.

Split conformal. $s_i = 1 - \hat{p}(y_i|x_i)$; $\hat{q} =$ the $\lceil (n+1)(1-\alpha) \rceil / n$ quantile; $C(x) = \{y : \hat{p}(y|x) \geq 1 - \hat{q}\}$; $\Pr[y \in C(x)] \geq 1 - \alpha$ under exchangeability.

Beam over a hierarchy (vendor cookbook). path score $= \big(\prod_{\text{edges}} p_e\big)^{1/\#\text{decisions}}$, keep the $K = 3$ best; the cookbook's "separation" diagnostic is the top path's score divided by the second's (near 1 means ambiguous).

Cascade cost (derived). per case $\approx c_{\mathrm{cheap}} + c_{\mathrm{verifier}} + f_{\mathrm{flag}}\, c_{\mathrm{expensive}}$ with f_{flag} the flagged fraction; the gate on the probability that a field is wrong is p^* for "trigger the re-extraction" (slide 18).

Batching (vendor cookbook). 13 questions in one call vs. 13 calls: \$0.000497 vs. \$0.00609 (12.2$\times$), 0.27 s vs. 2.71 s summed (10.0$\times$).

C Exercises

1. **Confidences.** Compute the Choice confidence for $(0.5, 0.3, 0.2)$, $(0.5, 0.25, 0.25)$ and $(0.34, 0.33, 0.33)$. Compute the Score (expected level) and the Score confidence for five levels with probabilities $(0.1, 0.2, 0.4, 0.2, 0.1)$ and $(0.45, 0, 0.1, 0, 0.45)$. Explain in one sentence why the second Score distribution gets confidence 0 although its argmax probability is 0.45.

2. **Two summaries.** For the pair $(0.6, 0.3, 0.1)$ and $(0.6, 0.2, 0.2)$, compute the vendor's confidence, the margin to the runner-up, and the entropy. Which summary would you threshold on for an action whose cost is dominated by picking the runner-up by mistake?

3. **A reliability diagram.** A model answered 1,000 questions. In the bins $[0.5, 0.6)$, $[0.6, 0.7)$, $[0.7, 0.8)$, $[0.8, 0.9)$, $[0.9, 1.0]$ it placed 150, 200, 250, 250, 150 answers with mean predicted probabilities 0.55, 0.65, 0.75, 0.85, 0.95 and accuracies 0.50, 0.52, 0.61, 0.70, 0.88. Draw the diagram, compute the ECE, and say whether the model is over- or underconfident. What single temperature direction ($T > 1$ or $T < 1$) would help?

4. **Proper scoring.** Show that the expected Brier score $p(1-q)^2 + (1-p)q^2$ is minimized at $q = p$ and that the expected log loss is too. Then propose a scoring rule that is *not* proper (other than 0/1 loss) and exhibit a p at which the optimal report is not p.

5. **Murphy.** A model always predicts the base rate 0.3 on a binary task whose true base rate is 0.3. Compute its Brier score, its reliability term and its resolution term. Compare with a model that predicts 0.9 on the 30% of cases that are positive and 0.1 on the rest.

6. **Temperature.** Logits $(3, 1, 0)$. Compute the softmax at $T = 1$ and find the T at which the top probability equals 0.6 (solve numerically). State what happens to the argmax and to the ranking.

7. **Thresholds from costs.** A fraud Noul guards a refund. A missed fraud costs \$200; a wrongly held legitimate refund costs \$8 of support time and goodwill. What is p^* for "hold the refund"? Recompute if a held legitimate refund also loses the customer with probability 0.1 at a cost of \$500.

8. **Selective prediction.** On 1,000 cases a model is wrong on 200. Sorting by confidence, the errors among the top 20%, 40%, 60%, 80% and 100% of cases are 2, 12, 40, 100 and 200. Draw the risk–coverage curve, estimate the AURC by the trapezoid rule (take the risk at zero coverage to be 0 and interpolate linearly between the listed points), and find the largest listed coverage at which the selective risk is at most 5%.

9. **Conformal.** Twenty calibration questions gave the true option these probabilities: 0.95, 0.91, 0.88, 0.85, 0.83, 0.80, 0.78, 0.75, 0.72, 0.70, 0.66, 0.62, 0.58, 0.55, 0.50, 0.44, 0.41, 0.35, 0.28, 0.20. For $\alpha = 0.1$, compute $\hat{q}$ and the probability floor $1 - \hat{q}$. For the three tickets on slide 19, which options are in each set? (The slide's picture used an illustrative $\hat{q} = 0.88$; your floor will differ.) What changes at $\alpha = 0.2$?

10. **Cascade economics.** A cheap extractor costs \$0.004 per document, the verifier \$0.0005, and the expensive re-extraction \$0.09. At a flag rate of 15%, what is the cost per document? At what flag rate does the cascade cost more than sending everything to the expensive model? If the verifier's flag threshold is lowered from 0.7 to 0.5, which direction does each of the flag rate, the cost and the error move?

11. **Batching.** The parallel-questions cookbook sent a 53,777-character article. Estimate its token count at 4 characters per token, the input cost at \$0.042 per million for one call and for 13 calls, and compare with the cookbook's \$0.000497 and \$0.00609. Explain any gap.

12. **Order sensitivity.** Design an experiment to measure a decision model's flip rate under option reversal and under random shuffle on your own 200-case set, including how you would average probabilities across rotations. What flip rate would you consider acceptable for an action with $p^* = 0.9$, and why?

13. **Injected context.** Take the slide-28 request and write three one-sentence additions to the `task` field that an attacker might use to raise `reversible` and `serves_task` above 0.9 for a destructive command. For each, say which harness-level defence (deny-list, review band, schema check) would stop it.

14. **Label versus outcome.** In Anaconda's CFPB study the zero-shot model scored 45% on real outcomes and 82% once rules were written into the question. Give two reasons the first number was below the majority baseline, and one reason the second number still overstates what the model knows.

15. **Build-your-own.** Sketch recipe 1 of slide 30 as pseudocode for an open model served with a logprobs API: prompt construction, option rotation, probability extraction, temperature fitting on a labelled set, and the output shape of a Choice answer. Then list the three rows of the slide-26 table you would run first and why.

D Reading list

The papers behind each part of the deck, in the order they are discussed, with one line on why each is worth reading. Entries with an arXiv identifier are included in the paper archive; entries without one are listed with a URL. Items marked *secondary source* are blog posts, model cards or commentary.

1. Overview and lineage (slides 1-4)

- **TypeSafe AI (2026).** *System One: fast, structured decisions for software (documentation)*. `https://docs.typesafe.ai/concepts/system-one`. The vendor's definition of a decision model: state in, typed answers with calibrated probabilities out, no text. Read first. Secondary source.
- **TypeSafe AI (2026).** *AI primer: why decision models are trained for calibrated probabilities (documentation)*. `https://docs.typesafe.ai/introduction/machine-learning-primer`. The vendor's contrast of RLHF, RLVR and its own RLCD, and its definition of calibration. The only public description of the training objective. Secondary source.
- **Kahneman (2011).** *Thinking, Fast and Slow*. `https://en.wikipedia.org/wiki/Thinking,_Fast_and_Slow`. The System 1 / System 2 metaphor the category borrows its name from. Book; secondary source for the deck's purposes.
- **Booch et al. (2020).** *Thinking fast and slow in AI*. arXiv:2010.06002. The AAAI framing of fast, automatic components beside slow, deliberate ones in AI systems; the intellectual ancestor of 'System One models'.
- **Yu, Xu, Weston and Kulikov (2024).** *Distilling System 2 into System 1*. arXiv:2407.06023. Compressing deliberate reasoning into direct answers; where it works and where (GSM8K chain of thought) it fails. The closest academic cousin of a decision model's promise.
- **Niculescu-Mizil and Caruana (2005).** *Predicting good probabilities with supervised learning*. `https://www.cs.cornell.edu/~alexn/papers/calibration.icml05.crc.rev3.pdf`. Which classical learners are calibrated out of the box and which need Platt or isotonic correction. Calibrated classification predates deep learning. ICML 2005.
- **Hendrycks et al. (2020).** *Measuring massive multitask language understanding*. arXiv:2009.03300. MMLU and the convention of scoring options by the log-probability of the answer letter: the first widely used 'decision readout' from a generative model.
- **Ouyang et al. (2022).** *Training language models to follow instructions with human feedback*. arXiv:2203.02155. InstructGPT: the reward model (a decision model in all but name) and the RLHF pipeline that later work shows erodes calibration. TypeSafe's CEO is a co-author.
- **Zheng et al. (2023).** *Judging LLM-as-a-judge with MT-Bench and Chatbot Arena*. arXiv:2306.05685. LLM judges agree with humans over 80% of the time and show position, verbosity and self-enhancement biases; the generative route to decisions and its known faults.
- **Kalai, Nachum, Vempala and Zhang (2025).** *Why language models hallucinate*. arXiv:2509.04664. Hallucination as classification error plus an evaluation culture that rewards guessing over abstention; the argument a calibrated, abstaining decision model answers.

2. Primitives, state and the API (slides 5-9)

- **TypeSafe AI (2026).** *Primitives: Choice, Score, Noul, Box (documentation)*. `https://docs.typesafe.ai/primitives`. The four question types, their answer shapes, and how to choose among them. Secondary source.
- **TypeSafe AI (2026).** *Confidence (documentation)*. `https://docs.typesafe.ai/confidence`. The exact confidence formulas for Choice and Score, the worked examples on slides 6-7, the three-band threshold advice. Secondary source.
- **TypeSafe AI (2026).** *State (documentation)*. `https://docs.typesafe.ai/concepts/state`. How to structure the content a question is asked about; facts in the state, judgments in the questions. Secondary source.
- **TypeSafe AI (2026).** *How to build with System One (documentation)*. `https://docs.typesafe.ai/concepts/how-to-build-with-system-one`. Code in control, narrow atomic questions, what/not_for/examples per option, batching, routing on confidence. Secondary source.
- **TypeSafe AI (2026).** *System One API reference and request rules (documentation)*. `https://docs.typesafe.ai/api/system-one`. Request and response shapes, limits (64k per request, 32k per question, 32 MiB body), headers for images and long context, status codes. Secondary source.
- **TypeSafe AI (2026).** *Cookbook: parallel questions (documentation)*. `https://docs.typesafe.ai/cookbooks/parallel_questions`. Thirteen questions over a 53,777-character article in one call versus thirteen calls: 12.2x cheaper, 10x faster, answers identical on 11 of 13 and equally noisy on the rest. The batching evidence on slide 9. Secondary source.
- **OpenRouter (2026).** *Decisions API: Jev and the decision-model category (documentation)*. `https://openrouter.ai/docs/guides/community/jev`. The cross-vendor decisions endpoint, the catalog of decision models, and the warning that thresholds do not transfer across models. Secondary source.
- **OpenAI (2026).** *Decisions API guide (documentation)*. `https://developers.openai.com/api/docs/guides/decisions`. OpenAI's version of the same primitives (predicate, choice, score, refusal) on gpt-6-luna; no calibration claim, thresholds to be set from your own labels. Secondary source.
- **Cheng, Kasai and Yu (2023).** *Batch prompting: efficient inference with large language model APIs*. arXiv:2301.08721. Many questions in one generative call: up to 5x token and time savings, with accuracy degrading as the batch grows. The academic precedent for fan-out, and the contrast with a decision model's independent questions.
- **Son et al. (2024).** *Multi-task inference: can large language models follow multiple instructions at once?*. arXiv:2402.11597. Evaluating several instructions in a single pass; speedups and the conditions under which quality holds.
- **Liu et al. (2023).** *G-Eval: NLG evaluation using GPT-4 with better human alignment*. arXiv:2303.16634. Scoring with the probability-weighted sum over rating tokens instead of the argmax: the same construction as the Score primitive's expected level.
- **Robinson, Rytting and Wingate (2022).** *Leveraging large language models for multiple choice question answering*. arXiv:2210.12353. Multiple-choice symbol binding: present the options jointly and read the symbol. The readout that a Choice question formalizes.

3. Calibration and proper scoring rules (slides 10-13)

- **Guo, Pleiss, Sun and Weinberger (2017).** *On calibration of modern neural networks*. arXiv:1706.04599. Modern deep networks are overconfident where 1990s networks were not; reliability diagrams, ECE, and temperature scaling. The paper every LLM calibration study cites. ICML 2017.
- **Gneiting and Raftery (2007).** *Strictly proper scoring rules, prediction, and estimation*. `https://doi.org/10.1198/016214506000001437`. The theory of scoring rules under which reporting the true belief is the unique optimum: why Brier and log loss can serve as training rewards. JASA 2007.
- **DeGroot and Fienberg (1983).** *The comparison and evaluation of forecasters*. `https://doi.org/10.2307/2987588`. The calibration-refinement decomposition and the reliability diagram. The Statistician 1983.
- **Platt (1999).** *Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods*. `https://www.researchgate.net/publication/2594015`. Platt scaling: a sigmoid fitted to classifier scores; temperature scaling is its one-parameter special case.
- **Nixon et al. (2019).** *Measuring calibration in deep learning*. arXiv:1904.01685. The choice of calibration metric re-ranks methods; adaptive binning, class conditioning and L2 variants of ECE. The pitfalls row on slide 26.
- **Kumar, Liang and Ma (2019).** *Verified uncertainty calibration*. arXiv:1909.10155. Scaling methods look better calibrated than they are; scaling-then-binning gives a verifiable estimate with known sample complexity.
- **Kull et al. (2019).** *Beyond temperature scaling: obtaining well-calibrated multiclass probabilities with Dirichlet calibration*. arXiv:1910.12656. Multiclass calibration as a linear map on log-probabilities, and class-wise ECE; relevant to Choice questions with many options.
- **Minderer et al. (2021).** *Revisiting the calibration of modern neural networks*. arXiv:2106.07998. Architecture is a major determinant of calibration; some recent architectures are among the best calibrated. Calibration is not a monotone function of size.
- **Blasiok, Gopalan, Hu and Nakkiran (2022).** *A unifying theory of distance from calibration*. arXiv:2211.16886. ECE is discontinuous; consistent, smooth distance-from-calibration measures. STOC 2023.
- **Ovadia et al. (2019).** *Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift*. arXiv:1906.02530. Post-hoc calibration degrades under distribution shift; the warning behind 'calibrate on your own data'.
- **Zhao, Wallace, Feng, Klein and Singh (2021).** *Calibrate before use: improving few-shot performance of language models*. arXiv:2102.09690. A content-free input exposes a label prior in LLM classifiers; correcting it adds up to 30 points of accuracy and cuts prompt variance.
- **Kadavath et al. (2022).** *Language models (mostly) know what they know*. arXiv:2207.05221. Large models are well calibrated on multiple-choice and true/false questions in the right format, can predict P(True) of their own samples, and lose calibration when the format shifts.
- **OpenAI (2023).** *GPT-4 technical report*. arXiv:2303.08774. Figure 8: the pre-trained model is well calibrated on an MMLU subset and the PPO-tuned model is not; the sentence 'post-training hurts calibration significantly'.

- **Lin, Hilton and Evans (2022).** *Teaching models to express their uncertainty in words*. arXiv:2205.14334. GPT-3 fine-tuned to state a verbal probability is calibrated and remains moderately calibrated under distribution shift; CalibratedMath.
- **Tian et al. (2023).** *Just ask for calibration: strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback*. arXiv:2305.14975. For RLHF models, verbalized confidence beats the conditional token probability, cutting ECE by up to two-thirds (gpt-3.5, TriviaQA: 0.140 to 0.050).
- **Xiong et al. (2023).** *Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs*. arXiv:2306.13063. A black-box elicitation framework (prompting, sampling, aggregation); LLMs remain overconfident and cluster their stated numbers.
- **Zhu et al. (2023).** *On the calibration of large language models and alignment*. arXiv:2311.13240. Pretraining improves calibration with scale; instruction tuning worsens it, more so with synthetic data.
- **Kapoor et al. (2024).** *Large language models must be taught to know what they don't know*. arXiv:2406.08391. Prompting is not enough; about a thousand graded examples and LoRA on internal features give uncertainty estimates that generalize.
- **Xiao et al. (2025).** *Restoring calibration for aligned large language models: a calibration-aware fine-tuning approach*. arXiv:2505.01997. Preference collapse as the mechanism by which alignment destroys calibration, and a fine-tuning objective that restores it.
- **Sanz-Guerrero and von der Wense (2026).** *Calibration as a first-class criterion in LLM evaluation*. arXiv:2609.26489. A position paper: none of nine recent model reports report calibration at all.
- **Wang et al. (2024).** *'My answer is C': first-token probabilities do not match text answers in instruction-tuned language models*. arXiv:2402.14499. For instruction-tuned models the first-token probability disagrees with the generated text answer at rates reaching over 60%: the central caveat for logprob readouts.

4. Training for calibration (slides 14-16)

- **Yang, Klein, Celikyilmaz, Peng and Tian (2023).** *RLCD: reinforcement learning from contrastive distillation for language model alignment*. arXiv:2307.12950. The other RLCD: preference pairs from positive and negative prompts, for alignment. Nothing to do with calibration; included so the name collision is not mistaken for a source. ICLR 2024.
- **Mielke, Szlam, Dinan and Boureau (2022).** *Reducing conversational agents' overconfidence through linguistic calibration*. arXiv:2012.14983. The earliest linguistic-calibration training signal for dialogue agents.
- **Band, Li, Ma and Hashimoto (2024).** *Linguistic calibration of long-form generations*. arXiv:2404.00474. Calibration defined through the decisions a reader makes from the text; supervised then RL training on a decision-based reward.
- **Stengel-Eskin, Hase and Bansal (2024).** *LACIE: listener-aware finetuning for confidence calibration in large language models*. arXiv:2405.21028. Calibration as preference optimization against a simulated listener; fewer wrong answers accepted by humans.
- **Xu et al. (2024).** *SaySelf: teaching LLMs to express confidence with self-reflective rationales*. arXiv:2405.20974. Supervised fine-tuning on self-reflective rationales followed by PPO with a quadratic confidence reward.

- **Leng, Huang, Zhu and Huang (2024).** *Taming overconfidence in LLMs: reward calibration in RLHF*. arXiv:2410.09724. Reward models prefer high stated confidence; two fixes inside the standard RLHF pipeline. Why preference rewards are not proper scoring rules.
- **Bani-Harouni, Pellegrini, Stangel et al. (2025).** *Rewarding doubt: a reinforcement learning approach to calibrated confidence expression of large language models*. arXiv:2503.02623. RL with a log-scoring-rule reward on stated confidence; the optimal policy is exactly calibrated. The cleanest statement of the principle behind a calibration objective.
- **Damani et al. (2025).** *Beyond binary rewards: training LMs to reason about their uncertainty*. arXiv:2507.16806. RLCR: correctness plus a Brier score on the verbalized confidence; ordinary RL hurts calibration, RLCR improves it at no accuracy cost. The nearest published analogue of the vendor's 'RL for calibrated decisions'.
- **Yoon et al. (2025).** *Reasoning models better express their confidence*. arXiv:2505.14489. Reasoning models are better calibrated in most settings tested, attributed to slow-thinking behaviours.
- **Mei et al. (2025).** *Reasoning about uncertainty: do reasoning models know when they don't know?*. arXiv:2506.18183. The counterpoint: reasoning models are overconfident on wrong answers, and deeper reasoning can make it worse.
- **Zeng, Xuan, Cui and Voigt (2025).** *Thinking out loud: do reasoning models know when they're right?*. arXiv:2504.06564. A 'reasoning tax': RL and distillation improve calibration on reasoning tasks but erode awareness of knowledge boundaries.
- **Kirichenko, Ibrahim, Chaudhuri and Bell (2025).** *AbstentionBench: reasoning LLMs fail on unanswerable questions*. arXiv:2506.09038. Twenty datasets and twenty models; fine-tuning for reasoning degrades abstention by about a quarter on average.
- **Hinton, Vinyals and Dean (2015).** *Distilling the knowledge in a neural network*. arXiv:1503.02531. Soft targets as the training signal; the original recipe for compressing a large model's probabilities into a small one.
- **Hsieh et al. (2023).** *Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes*. arXiv:2305.02301. A 770M model trained on a large model's rationales beats 540B few-shot with a fraction of the data; distillation into a decision-sized model.
- **Pangakis and Wolken (2024).** *Knowledge distillation in automated annotation: supervised text classification with LLM-generated training labels*. arXiv:2406.17633. Classifiers trained on GPT-4 labels match those trained on human labels across 14 tasks; the cheapest route to a private decision model.

5. Selective prediction, thresholds, conformal sets, cascades (slides 17-20)

- **El-Yaniv and Wiener (2010).** *On the foundations of noise-free selective classification*. `https://jmlr.org/papers/v11/el-yaniv10a.html`. The risk-coverage formalism: a classifier paired with a selector, and the trade-off curve. JMLR 2010.
- **Geifman and El-Yaniv (2017).** *Selective classification for deep neural networks*. arXiv:1705.08500. Guaranteed risk at a chosen coverage by thresholding confidence; the practical recipe behind 'answer when sure, hand off the rest'.
- **Geifman, Uziel and El-Yaniv (2019).** *Bias-reduced uncertainty estimation for deep neural classifiers*. arXiv:1805.08206. The source of AURC, the area under the risk-coverage curve.

- **Hendrycks and Gimpel (2016).** *A baseline for detecting misclassified and out-of-distribution examples in neural networks*. arXiv:1610.02136. Maximum softmax probability as an error detector, scored by AUROC: ranking quality as distinct from calibration.
- **Madras, Pitassi and Zemel (2018).** *Predict responsibly: improving fairness and accuracy by learning to defer*. arXiv:1711.06664. Rejection as learning to defer to a downstream decision maker whose own error matters.
- **Mozannar and Sontag (2020).** *Consistent estimators for learning to defer to an expert*. arXiv:2006.01862. A reduction of deferral to cost-sensitive learning with a consistent surrogate loss.
- **Kamath, Jia and Liang (2020).** *Selective question answering under domain shift*. arXiv:2006.09462. Softmax-based abstention fails out of domain; a trained calibrator answers more questions at the same accuracy.
- **Elkan (2001).** *The foundations of cost-sensitive learning*. `https://cseweb.ucsd.edu/~elkan/rescale.pdf`. The optimal decision threshold from the cost matrix; learn probabilities, then decide. The formal justification for slide 18. IJCAI 2001.
- **Vickers and Elkin (2006).** *Decision curve analysis: a novel method for evaluating prediction models*. `https://doi.org/10.1177/0272989X06295361`. Net benefit as a function of the threshold probability: the range of thresholds over which a predictor helps at all.
- **Angelopoulos and Bates (2021).** *A gentle introduction to conformal prediction and distribution-free uncertainty quantification*. arXiv:2107.07511. The four-line split-conformal recipe on slide 19, its finite-sample guarantee, and its extensions.
- **Kumar et al. (2023).** *Conformal prediction with large language models for multi-choice question answering*. arXiv:2305.18404. Conformal sets over LLM option probabilities; set size tracks accuracy and coverage is approximately maintained under mild subject shift.
- **Quach et al. (2023).** *Conformal language modeling*. arXiv:2306.10193. Calibrated stopping and rejection rules that yield sets of generations containing an acceptable answer with high probability.
- **Ren et al. (2023).** *Robots that ask for help: uncertainty alignment for large language model planners*. arXiv:2307.01928. KnowNo: conformal prediction over multiple-choice next steps tells a robot exactly when to ask a human. A decision-model pattern in robotics.
- **Mohri and Hashimoto (2024).** *Language models with conformal factuality guarantees*. arXiv:2402.10978. Back off sub-claims until the retained text meets a correctness target; conformal guarantees for generated text.
- **Chen, Zaharia and Zou (2023).** *FrugalGPT: how to use large language models while reducing cost and improving performance*. arXiv:2305.05176. Cascades of LLMs with a learned scorer: GPT-4 quality at up to 98% lower cost, or higher accuracy at equal cost.
- **Jitkrittum et al. (2023).** *When does confidence-based cascade deferral suffice?*. arXiv:2307.02764. Confidence deferral is Bayes-optimal only when the large model's error is roughly constant; otherwise learn the deferral rule from both models.
- **Yue et al. (2023).** *Large language model cascades with mixture of thought representations for cost-efficient reasoning*. arXiv:2310.03094. Answer consistency as a difficulty signal for cascading; GPT-4 performance at a fraction of the cost.

- **Ong et al. (2024).** *RouteLLM: learning to route LLMs with preference data*. arXiv:2406.18665. Routers trained on preference data cut cost more than twofold at matched quality and transfer across model pairs.
- **Gupta et al. (2024).** *Language model cascades: token-level uncertainty and beyond*. arXiv:2404.10136. Sequence-probability deferral has a length bias; quantile features and learned deferral do better.
- **Chuang et al. (2024).** *Learning to route LLMs with confidence tokens*. arXiv:2410.13284. Learned confidence tokens beat verbalized and logit confidence for routing and rejection.
- **TypeSafe AI (2026).** *Cookbook: structured-data-extraction cascade (documentation)*. `https://docs.typesafe.ai/cookbooks/sde_cascade`. A cheap extractor, per-field Noul verification, and an expensive re-extraction only when flagged; the cascade on slide 20. Vendor numbers. Secondary source.
- **TypeSafe AI (2026).** *Pattern: confidence-gated routing (documentation)*. `https://docs.typesafe.ai/patterns/confidence-routing`. The three-band routing example (0.6 floor, 0.85 for transfers) on slide 18. Secondary source.

6. The landscape and open-weight decision models (slides 21-23)

- **TypeSafe AI (2026).** *Models and aliases (documentation)*. `https://docs.typesafe.ai/models`. Jev 1.13 and 1.14, prices, context limits, the long-context header, and alias behaviour. Secondary source.
- **Cloudflare (2026).** *Clef: open decision models (blog and model card)*. `https://blog.cloudflare.com/clef-decision-models/`. A frozen Qwen backbone with a joint schema head and LoRA, trained with cross-entropy plus Brier plus an ordinal term; the most detailed public architecture in the category. Apache-2.0 weights at huggingface.co/Cloudflare/clef. Secondary source.
- **Perplexity AI (2026).** *pplx-decider-v1.2 (model card)*. `https://huggingface.co/perplexity-ai/pplx-decider-v1.2`. A 27B backbone with a separate readout head read at a marker token after each question, non-causal attention and a fitted temperature; reports dev-set ECE 0.014. Apache-2.0. Secondary source.
- **Palmer (2026).** *Kev: an open reimplementation of a decision model (model card and README)*. `https://github.com/jaredpalmer/kev/blob/main/README.md`. A pointer head on a frozen 4B base with a rank-16 LoRA, trained on public datasets reshaped into typed requests; honest comparisons with Jev. Secondary source.
- **Convai Innovations (2026).** *Laya: a ModernBERT-based decision model (model card)*. `https://huggingface.co/convaiinnovations/laya`. A 421M encoder with a two-layer head scoring each option at its own mask token; the encoder route, with vendor and independent numbers that disagree. Secondary source.
- **ggml-org (2026).** *Decision models in llama.cpp (Hugging Face blog)*. `https://huggingface.co/blog/ggml-org/decision-models-in-llamacpp`. The /v1/systemone endpoint in llama.cpp and the table of open decision models with per-question latencies. Secondary source.
- **Microsoft (2026).** *Microsoft-Decision-1 (announcement)*. `https://commandline.microsoft.com/microsoft-decision-1-model-foundry/`. A post-trained 9B model with single-pass scoring and a 36-benchmark comparison; the vendor's robustness claims. Secondary source.
- **Liquid AI (2026).** *d1-ultra (blog)*. `https://www.liquid.ai/blog/d1-ultra`. One forward pass, no tokens generated, billed per question; the open d1-3B and d1-omni-600M siblings. Secondary source.

- **Li and Wagle (2026).** *LLM-as-Jev: LLMs are already Jev-style decision models; when and how to fine-tune them*. arXiv:2610.02076. Sum the option-suffix logprobs after a prefilled 'Best answer: [' and softmax: a 4B model reaches 81.4% on JevBench at ECE 0.057 with no training, 84.0% with LoRA.
- **Warner et al. (2024).** *Smarter, better, faster, longer: a modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference*. arXiv:2412.13663. ModernBERT: the encoder backbone behind Laya and the fine-tuned-encoder recipe on slide 30.
- **Tunstall et al. (2022).** *Efficient few-shot learning without prompts*. arXiv:2209.11055. SetFit: a contrastively tuned sentence encoder plus a head, competitive with far larger prompted models from a handful of labels.
- **Raschka (2026).** *Classifier history and Jev (newsletter)*. `https://magazine.sebastianraschka.com/p/classifier-history-and-jev`. A historian's view: 25,000 IMDb reviews through Jev for $0.65, and a fine-tuned ModernBERT that matches it. Secondary source.

7. Limits: jaggedness, order, injection, distribution (slides 24-25)

- **TypeSafe AI (2026).** *Model jaggedness: Jev 1.13 (documentation)*. `https://docs.typesafe.ai/model-jaggedness/jev-1.13`. The vendor's own list of rough edges: literal reading, counting and dates, negation, irrelevant context, adversarial content, structural invariants. Secondary source.
- **Zheng, Zhou, Meng, Zhou and Huang (2023).** *Large language models are not robust multiple choice selectors*. arXiv:2309.03882. Selection bias is mostly token bias on the option identifiers; PriDe debiases without labels. ICLR 2024.
- **Pezeshkpour and Hruschka (2023).** *Large language models sensitivity to the order of options in multiple-choice questions*. arXiv:2308.11483. Reordering options swings performance by 13 to 75 percent; calibration recovers part of it.
- **Wang et al. (2023).** *Large language models are not fair evaluators*. arXiv:2305.17926. Swapping candidate positions flips judge verdicts; balanced-position calibration as the fix.
- **Xu et al. (2026).** *JevOut: natural context can flip decision models*. arXiv:2609.30243. Fluent, natural-looking context additions flip 61-73% of initially correct decisions across four decision systems and seven datasets.
- **Hu et al. (2026).** *JevAdvBench: a benchmark and black-box attacks for reinforcement learning for calibrated decisions models*. arXiv:2609.31142. 9,744 single-edit attacks; an appended unverified opinion flips 12.1% of decisions and drags 38% of confident answers below a 0.8 review threshold.
- **Anaconda (2026).** *Decision time: testing a decision model on real complaint outcomes (blog)*. `https://www.anaconda.com/blog/decision-time`. CFPB complaints scored against what actually happened; order and repeat-call stability measured; a classical model wins on the real outcome. Secondary source.
- **Sharma, Duncan, Marceau Caron and Vollmer (2026).** *TypedBench: a benchmark for calibration, framing sensitivity, and cost in System One decision models*. arXiv:2610.11392. Five hundred support tickets, accuracy varying from 0.708 to 0.946 across paraphrases of the same question, systematic underconfidence, and the cost of acting on miscalibrated probabilities.
- **Molas (2026).** *Jev can't be calibrated (blog)*. `https://alexmolas.com/2026/09/23/jev-cant-be-calibrated.html`. Calibration is relative to a distribution of questions; a fair coin asked of a decision model gets 0.92. Secondary source.

8. Evaluation and the independent evidence (slides 26-27)

- **Rafe and Das (2026).** *Calibrated decisions at scale: converting police crash narratives into probabilistic crash variables with a System One model (Jev)*. arXiv:2609.24052. 195,857 narratives and 27 questions for $0.154 per thousand; F1 0.908 against 2,416 human labels; raw probabilities overstate prevalence until recalibrated. The first independent at-scale study.
- **Guo et al. (2026).** *Just ask Jev: reinforcement learning for calibrated decisions as a zero-shot detector of AI alignment failures*. arXiv:2609.29429. Forty-four benchmarks: a strong ranker (median AUROC 0.886) at a fraction of an LLM judge's cost, with calibration error worse than a null baseline.
- **Wu and Lim (2026).** *REFLEX with Jev: efficient selective control in LLM agents*. arXiv:2609.26532. A decision model as the gate in an agent: 95% task success with 72.7% fewer strong-model calls, and little gain where cheap cascades already route well.
- **OpenRouter (2026).** *Jev vs Claude Opus 5 on Banking77 classification (blog)*. `https://openrouter.ai/blog/insights/jev-vs-claude-opus-5-classification/`. 3,080 utterances and 77 intents: 81.0% against 84.4%, latency percentiles, cost per thousand, and a confidence cascade that closes the gap. Secondary source.
- **OpenRouter (2026).** *Jev vs LLM-as-a-judge (blog)*. `https://openrouter.ai/blog/tutorials/jev-vs-llm-as-a-judge/`. 138 faithfulness and summary-quality items: accuracy, Brier score, latency and cost against a generative judge, and where the open rubric favours the generator. Secondary source.
- **Archestra (2026).** *We tested Jev on 100 real agent calls (blog)*. `https://archestra.ai/blog/we-tested-jev-on-100-real-agent-calls`. Tool-call approval decisions from a coding agent: zero-shot and few-shot accuracy against a frontier model and the majority baseline, with an order-reversal test. Secondary source.
- **Dibia (2026).** *How Jev works: calibrated decisions and a DIY baseline (newsletter)*. `https://newsletter.victordibia.com/p/how-jev-works-calibrated-decision`. The option-scoring hypothesis, and a 7B logprob readout with a temperature that lands within a few points of Jev on prompt-injection detection. Secondary source.
- **Lambert et al. (2024).** *RewardBench: evaluating reward models for language modeling*. arXiv:2403.13787. Reward models evaluated as classifiers over chosen and rejected responses; the evaluation culture decision models inherit.
- **Galil, Dabbah and El-Yaniv (2023).** *What can we learn from the selective prediction and uncertainty estimation performance of 523 ImageNet classifiers?*. arXiv:2302.11874. Selective-prediction metrics at scale: AURC can hide differences; which training regimes give the best selectors.
- **Gilardi, Alizadeh and Kubli (2023).** *ChatGPT outperforms crowd-workers for text-annotation tasks*. arXiv:2303.15056. The annotation-economics baseline: LLM labels at a fraction of crowd cost, before decision models cut it further.

9. Practice: agents, questions, building your own (slides 28-30)

- **OpenRouter (2026).** *Cookbook: auto-approve permission prompts with Jev (documentation)*. `https://openrouter.ai/docs/cookbook/coding-agents/auto-approve-permission-prompts-with-jev`. Two Noul questions, a 0.9 threshold, a static deny-list in front,

and fail-closed timeouts, wired into Claude Code, Codex, OpenCode and Cursor hooks. The pattern on slide 28. Secondary source.

- **TypeSafe AI (2026).** *Jev with coding agents (documentation).* `https://docs.typesafe.ai/introduction/coding-agents`. What a decision model is and is not inside a coding agent; the agent skill for writing integrations. Secondary source.
- **LangChain (2026).** *Building a harness with Jev (blog).* `https://www.langchain.com/blog/building-a-harness-with-jev`. A classifier, a model-router middleware and a tool guardrail built on a decision model; experimental. Secondary source.
- **TypeSafe AI (2026).** *Cookbook: guardrails for LLMs (documentation).* `https://docs.typesafe.ai/cookbooks/llm_guardrails`. Input and output screening with four Nouls and a severity Score per message, two thresholds per hazard, and a precedence rule. Secondary source.
- **TypeSafe AI (2026).** *Cookbook: hierarchical classification (documentation).* `https://docs.typesafe.ai/cookbooks/hierarchical_classification`. Beam search over Choice probabilities through deep taxonomies, scored by the geometric mean of edge probabilities. Secondary source.
- **Inan et al. (2023).** *Llama Guard: LLM-based input-output safeguard for human-AI conversations*. arXiv:2312.06674. A generative safety classifier with a customizable taxonomy; the guard-model lineage a Noul-based guardrail replaces.
- **Bucher and Martini (2024).** *Fine-tuned 'small' LLMs (still) significantly outperform zero-shot generative AI models in text classification.* arXiv:2406.08660. When you have labels, a fine-tuned small model beats a prompted large one; the case for recipe 3 on slide 30.

Index of terms

0/1 loss, 16
529 Overloaded, 29

AbstentionBench, 22
adaptive binning, 36
agent loop, 6
alias, 14
AnyJev, 32
argmax, 5
Artificial Analysis Decision Index, 29
assets, 13
atomic question, 14
AURC, 24
AUROC of confidence, 24

backticked path, 14
Banking77, 38
base rate, 15
base-rate blindness, 35
Bayes action, 36
beam search over Choice probabilities, 10
binary classification, 9
blast radius, 39
block-causal mask, 31
Box, 9
Brier score, 15

calibrated, 4
calibration set, 23
calibration-aware fine-tuning, 18
cascade, 8
CFPB complaints, 38
chain of thought, 21
Choice, 5
class-wise ECE, 23
Cohen's κ, 36
composite scoring, 12
confidence, 4
confidence bands, 25
confidence floor, 26
conformal prediction, 27
context limit, 14
context rot, 14
contrastive distillation (RLCD of Yang et al.), 20
cost-sensitive learning, 25
cost-weighted error, 36
coverage, 24
criteria, 5
cross-entropy training, 16

decision head, 19
decision log, 39
decision model, 4
decisions endpoint, 30
decomposition, 40
deferral rule, 28
deny-list, 39
difficulty signal, 28
Dirichlet calibration, 23
distillation from a judge, 41
double negative, 34

entropy, 10
escalation, 7
exchangeability, 27
exhaustive options, 9
expected calibration error (ECE), 15
expected level, 12

fail closed, 39
false positive / false negative, 25
few-shot (decision model), 38
first-option bias, 10
first-token mismatch, 19
flag threshold, 28
flip rate, 34
format shift, 18
FrugalGPT, 28

G-Eval, 8
generative model, 5
GGUF, 32
guard, 6
guard model, 8
guessing incentive, 17

HaluEval / SummEval, 38
held-out set, 36
hook, 39

independent evaluation, 14
instructions, 5

isotonic regression, 23

jaggedness, 33
Jev, 4
Jev Decision Index / S1MB, 32
JevAdvBench, 35
JevOut, 35
joint schema head, 31

Kev, 32
KnowNo, 27

label prior / contextual calibration, 18
LACIE, 20
Laya, 32
learning to defer, 24
legend, 11
level, 11
literal reading, 33
LLM-as-a-judge, 8
LLM-as-Jev, 32
load shedding, 29
log loss, 15
logprob readout, 8
logprobs API, 19
LoRA, 41

macro-F1, 36
majority baseline, 36
margin, 10
marginal coverage, 27
marker token, 41
mean absolute deviation, 12
miscoverage level α, 27
mode, 12
ModernBERT, 32
multiclass classification, 9
Murphy decomposition, 15

net benefit, 25
non-causal attention, 31
nonconformity score, 27
Noul, 6
Noul certainty, 13
null baseline (calibration), 38

OpenJev, 32
OpenRouter, 29
opt-in header, 9
optimal threshold p^*, 25
option rotation, 41
order sensitivity, 34
ordinal partial credit, 41
ordinal regression, 9
outcome label, 35
output tokens, 5
overconfidence / underconfidence, 15

P(True), 18
p50 / p95 / p99, 29
parallel sampler, 20
permission prompt, 39
Platt scaling, 23
pointer head, 31
post-hoc calibration, 23
predicate, 30
prediction set, 27
preference collapse, 18
preference reward, 17
prefill-only inference, 41
presence check, 13
PriDe, 19
probability-weighted score, 12
prompt injection, 35
proper / strictly proper, 16

RAIL-M licence, 31
readout head, 31
reasoning tax, 22
refusal, 31
reject option, 24
reliability diagram, 15
review band, 35
review threshold / action threshold, 13
reward model, 8
Rewarding Doubt, 20
risk–coverage curve, 24
RLCD, 8
RLCR, 20
RLHF, 21
RLVR, 21
RouteLLM, 28
router, 8
routing, 6

sampling consistency, 19
SaySelf, 20
schema check, 40

Score, 7
scoring rule, 16
selective classifier, 24
selective risk, 24
SetFit, 41
sharpness, 15
Spearman correlation, 38
speculative fan-out, 14
state, 4
structural invariant, 34
sub-question ensemble, 40
sycophancy, 21
synthetic training data, 20
System One API, 5
System One model, 4

temperature T, 23
temperature scaling, 8
token bias / selection bias, 19
typed question, 4
TypedBench, 34
TypeSafe AI, 29

uniform distribution, 10
unsure band, 13
untrusted input, 40

verbalized confidence, 18
verification, 6
verified cascade, 28
version pinning, 40

what / not_for / examples, 10
wording sensitivity, 34

zero-mass failure, 35